DFCS Bootcamp – Day 5 Homework (Web App Pentesting & Recon)

Name: MUHAMMAD ZAIN NASIR

Program: Digital Forensics & Cyber Security Bootcamp – PSEBCYBERISB02

Trainer: Waseem Abbas Khan

Submitted: Wednesday 11 November 2026

Repository: https://github.com/the-raven-zn/dfcs-bootcamp/tree/main/s09-web-recon

Contents: Task 1 (Scope & Rules of Engagement) · Task 2 (Google Dork Report) · Task 3 (Subdomain Reconciliation) · Task 4 (Scanner Comparison: ZAP vs Burp Suite)

Method. This assignment covers all four tasks from the Day 5 handout: a scope document and rules of engagement for a hypothetical **.intel.com* engagement (Task 1); a passive Google dork report against the intentionally vulnerable *vulnweb.com* (Task 2); subdomain enumeration and reconciliation of *vulnweb.com* with three independent Kali tools, followed by DNS resolution of the combined list (Task 3); and a side-by-side comparison of OWASP ZAP and Burp Suite Community Edition against Metasploitable 2 / DVWA at Security Level: Low (Task 4). All active scanning in Task 4 was performed on an isolated host-only network between two VMs under my control; all reconnaissance in Tasks 2 and 3 was passive, read-only lookups against public indexes and DNS, with no exploitation of any kind performed.

Summary of tasks

#	Task	Target / Scenario	Weight	Headline result
1	Scope & Rules of Engagement	Hypothetical `*.intel.com` grey-box test	25%	Scope, named exclusion, break-glass clause, emergency stop procedure
2	Google Dork Report	`vulnweb.com` (passive)	25%	6 dorks with exposures + 2 negative results
3	Subdomain Reconciliation	`vulnweb.com` (passive, 3 tools)	25%	100 unique names; 8 resolve; 3 live (HTTP 200)
4	Scanner Comparison	Metasploitable 2 / DVWA (isolated lab)	25%	ZAP: 19 alert types; Burp manual: SQLi + command injection confirmed

Task 1 – Scope Document and Rules of Engagement

Scenario: a hypothetical web application penetration test of `*.intel.com`, conducted under written authorisation from the client. The engagement is **grey-box**: testers are supplied with two sets of test credentials, one per user role, so that authenticated functionality is reached rather than only the public login page.

1.1 – Scope

Section	Content
In scope	**`*.intel.com`** – all subdomains, current and newly discovered during the engagement window · IP range `198.51.100.0/24` (placeholder, replaced with the client's published range in a real engagement) · all REST and internal APIs reachable under the above domains
Explicitly out of scope	`career.intel.com` – third-party-hosted recruiting platform, excluded by name at the client's request · any third-party service or CDN endpoint merely pointed at the intel.com zone (e.g. a vendor's shared infrastructure), unless separately authorised · physical security and social engineering (see Prohibited Techniques)
Access granted	Two test accounts, one per application role (standard user, administrator), provisioned before testing begins · VPN access to any internal-only staging environments in scope · jump-server details (hostname, port, credentials) where applicable · box classification: **grey-box**

1.2 – Break-Glass Clause for Wildcard Discoveries

Because `*.intel.com` is a wildcard scope, it may include subdomains unknown to the client at contract signing, or created during the test window. Before testing any newly discovered host not already confirmed in scope:

- The tester contacts the named Engagement Point of Contact (see Testing Window table below).
- Written authorisation (email is sufficient) is received and logged before active testing begins on that host.
- The authorisation is appended to the engagement log and included in the final report.

1.3 – Prohibited Techniques

- No denial-of-service testing, or any technique likely to degrade production availability.
- No social engineering (phishing, pretexting, physical intrusion) unless separately scoped and authorised in writing.
- No destructive actions – no deleting, encrypting or corrupting data, under any circumstance.
- No testing outside the agreed testing window without prior written authorisation.

1.4 – Testing Window and Emergency Contact

Field	Value
Testing dates	[Start date] – [End date]
Testing hours	Business hours only, [time zone], to limit production impact
Engagement Point of Contact	[Name], [Direct phone], [Email] – authorises break-glass requests and receives critical-finding notifications
Emergency stop procedure	Any tester who believes testing caused unintended impact halts all activity immediately, notifies the Point of Contact by phone, and documents the incident within one hour

1.5 – Rules of Engagement

Rule	Why it exists
Written authorisation before the first packet	Signed by an authorised client representative; without it, testing is a criminal offence, not an assessment.
Test only what is listed in §1.1	Subject to the break-glass clause in §1.2 for wildcard discoveries; everything else belongs to someone else.
Data minimisation	If customer or personal data is encountered (e.g. via SQL injection or an exposed file), testers access and document the minimum necessary to prove impact – one record, one file – and never bulk-copy or exfiltrate a data store.
No destructive actions, authorised or not	Not permitted at any point without separate written sign-off naming the specific action.
Stop on critical finding	Any Critical-severity finding (e.g. unauthenticated remote code execution, full database compromise) is reported to the Point of Contact within 24 hours, rather than held for the final report.
Prompt reporting	Findings are delivered in the final report within X business days of testing completion.
Data destruction	Any test data, credentials, or copies of client data obtained during testing are destroyed within 30 days of report delivery, with written confirmation sent to the client.

Task 2 – Google Dork Report

Target: `vulnweb.com` – a deliberately vulnerable demonstration site published by Acunetix for security training. All queries below were run passively against Google's existing index. No page was opened, logged into, tested or exploited. `vulnweb.com` is intentionally vulnerable, so the exposures below are expected; they are documented here, with remediation, exactly as they would be for a real client engagement.

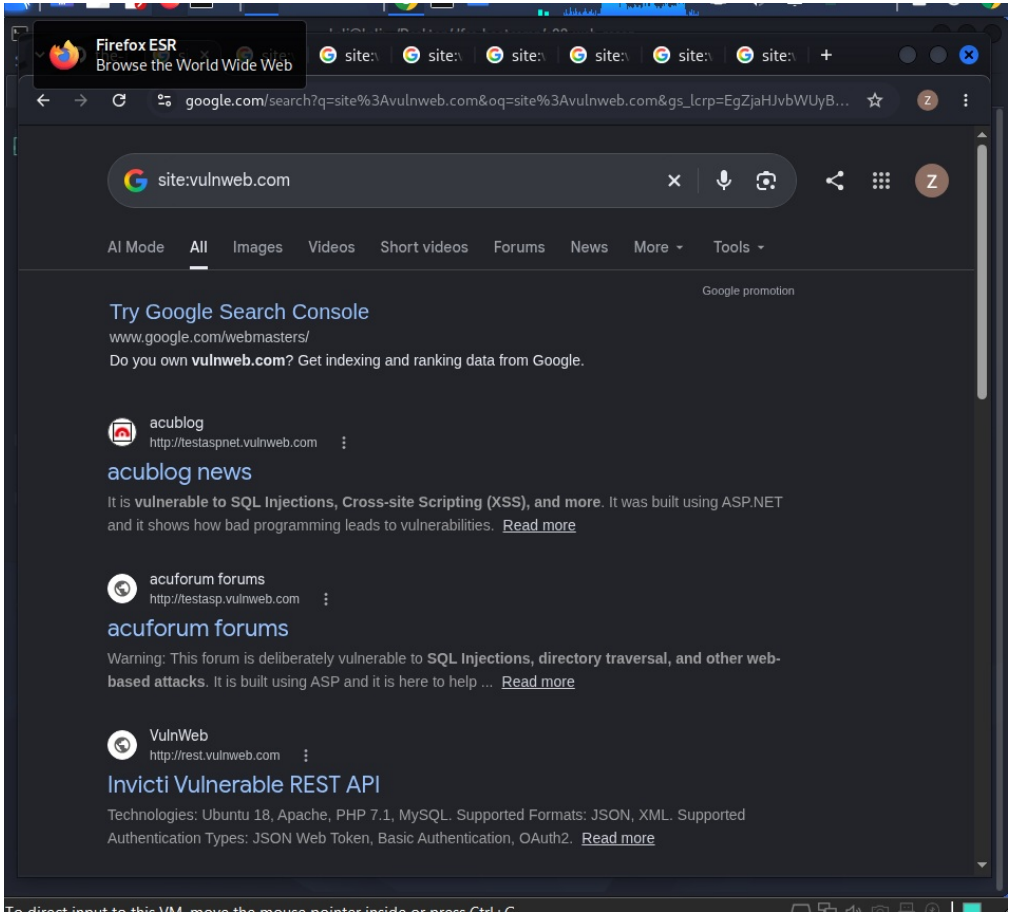
2.1 – Dork Table

#	Dork	What it found	Why it matters	Fix
1	`site:vulnweb.com -www`	Subdomains `rest`, `testasp`, `testaspnet` indexed, with deep paths (`/about`, `/showthread`, `/Comments`)	Maps hosts and pages without sending a single packet to the target; matches the subdomain enumeration in Task 3	noindex public hosts, Search Console removal requests, authentication/VPN in front of staging and test hosts
2	`site:vulnweb.com`	The `rest` listing's own snippet discloses: Ubuntu 18, Apache, PHP 7.1, MySQL, supported auth types JWT, Basic, OAuth2	Technology and version disclosure narrows an attacker's research to known CVEs; PHP 7.1 is end-of-life	Remove stack details from page descriptions/metadata; `ServerTokens Prod`; `ServerSignature Off`; patch unsupported software
3	`site:vulnweb.com inurl:admin`	`https://testphp.vulnweb.com/admin/` indexed	An administrative interface should never be discoverable via search	Restrict by IP/VPN, require MFA, noindex publicly
4	`site:vulnweb.com inurl:login`	Login pages indexed on `testphp`, `testasp` (acuforum), `testaspnet` (acublog)	Each page is a ready-made target for credential attacks	noindex, rate limiting, account lockout, MFA
5	`site:vulnweb.com filetype:php`	`signup.php` and `listproducts.php` on `testphp` indexed	Exposes dynamic script entry points; parameterised scripts like `listproducts.php` are classic injection candidates	Parameterised queries, input validation, clean URLs instead of raw script names, WAF
6	`site:vulnweb.com intitle:"index of"`	Exact phrase: no match. Fallback (unquoted) surfaced `testphp.vulnweb.com/pictures/` as a candidate directory path (unverified – not opened)	A bare directory path is a candidate for directory listing	`Options -Indexes` in Apache; add an index file; store uploads outside the web root

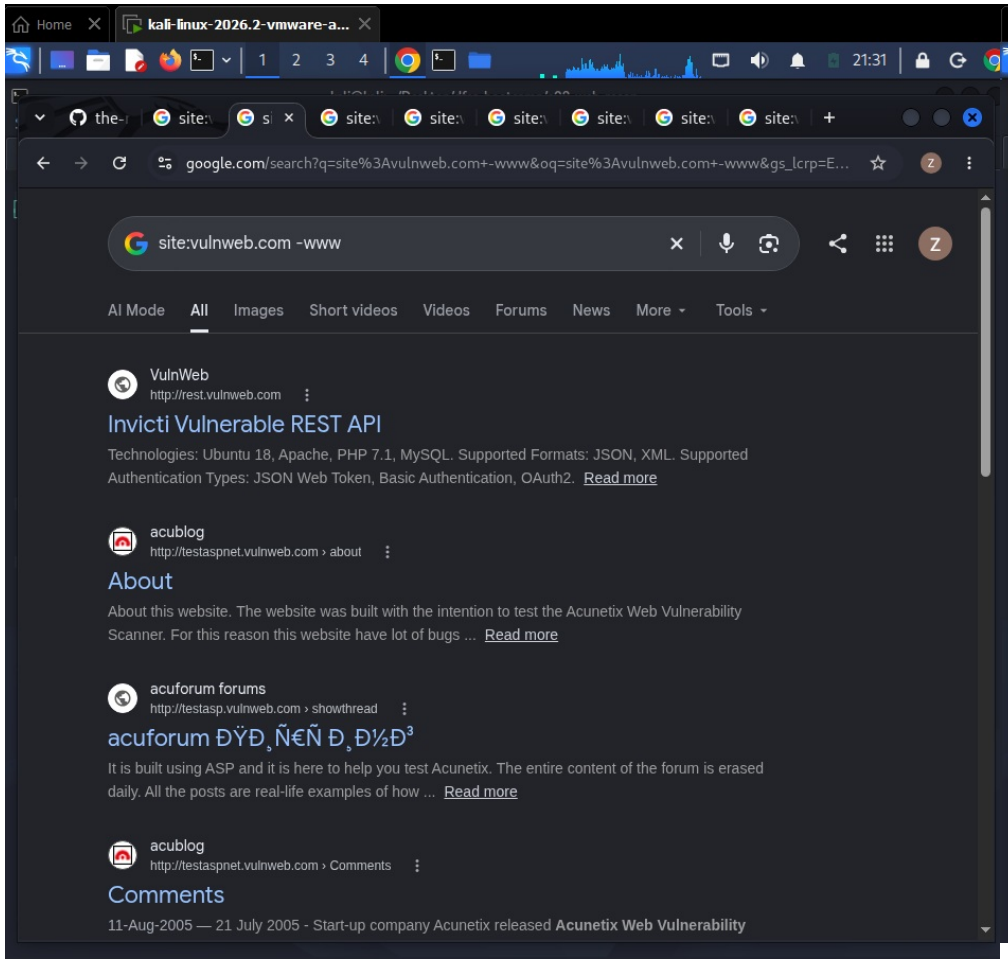
Negative results (also evidence):

- `site:vulnweb.com intext:"sql syntax"` – **no matches.** No database error messages are currently indexed.
- `site:vulnweb.com ext:sql OR ext:bak OR ext:log` – **no matches.** No backup, dump, or log files are currently indexed.

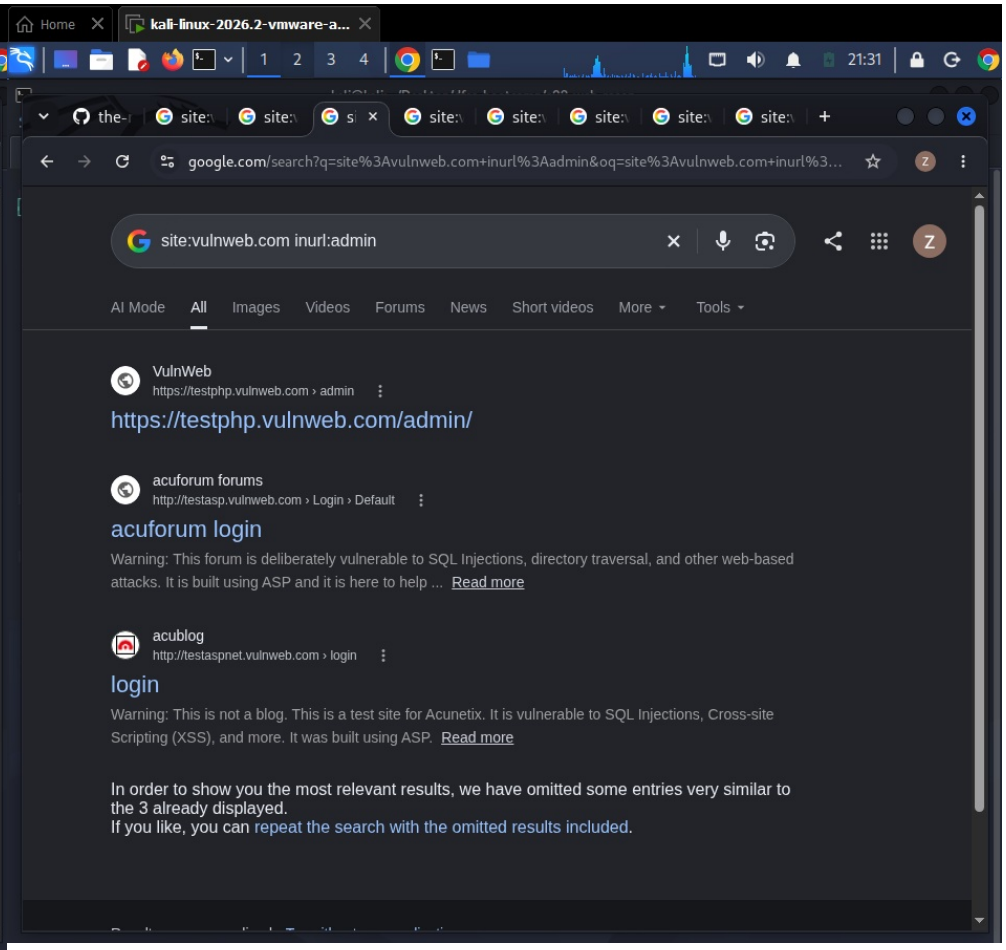
2.2 – Screenshots



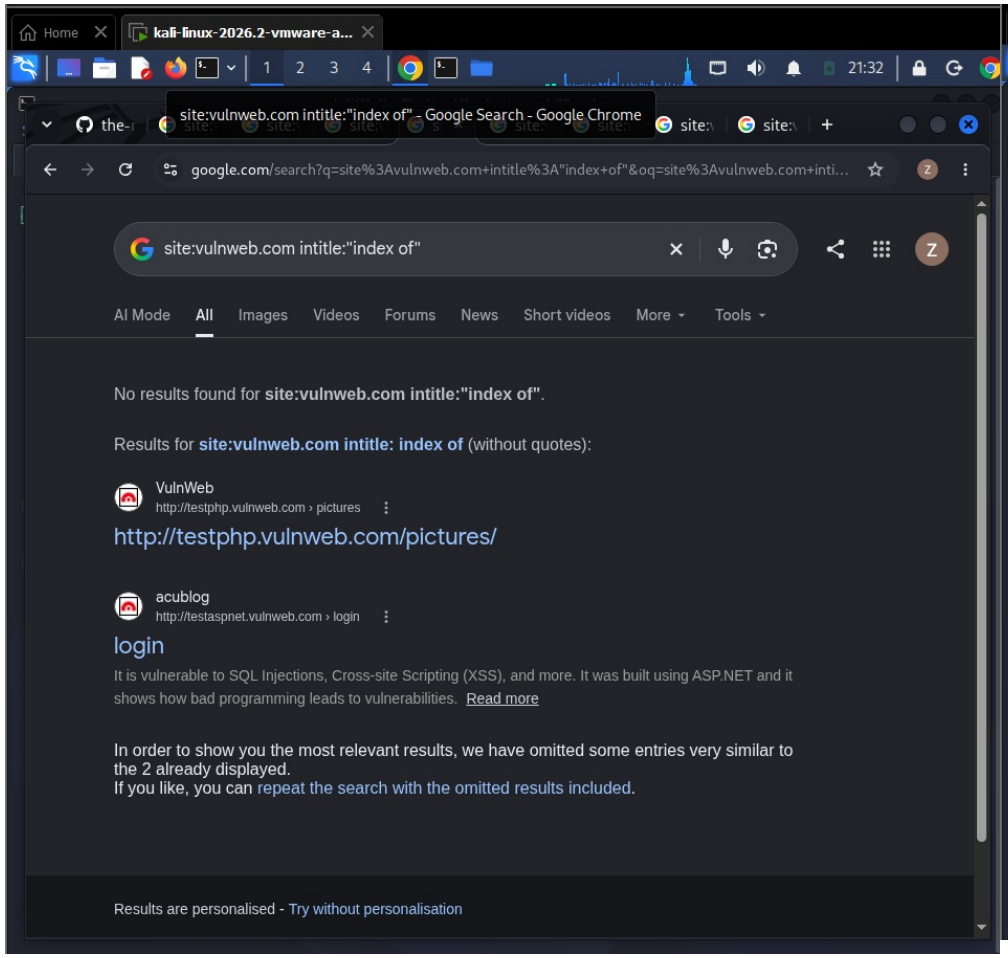
Dork 1: `site:vulnweb.com`



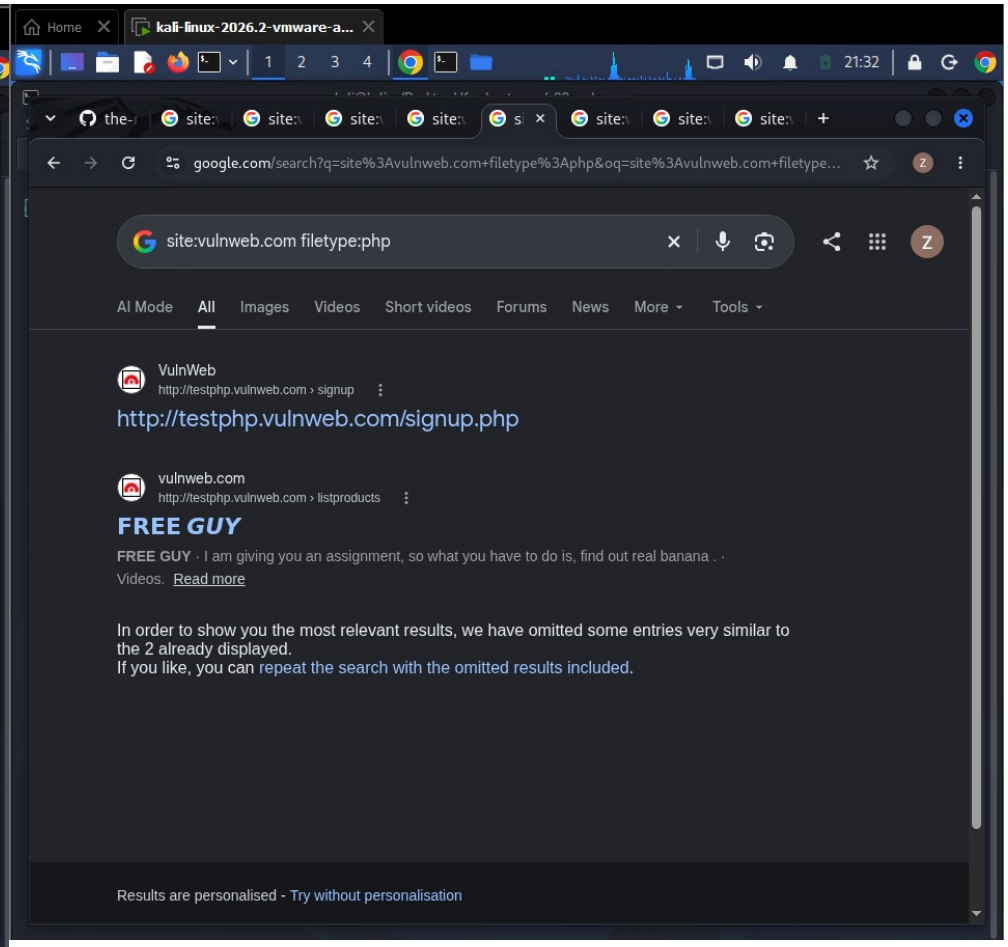
Dork 2: `site:vulnweb.com -www`



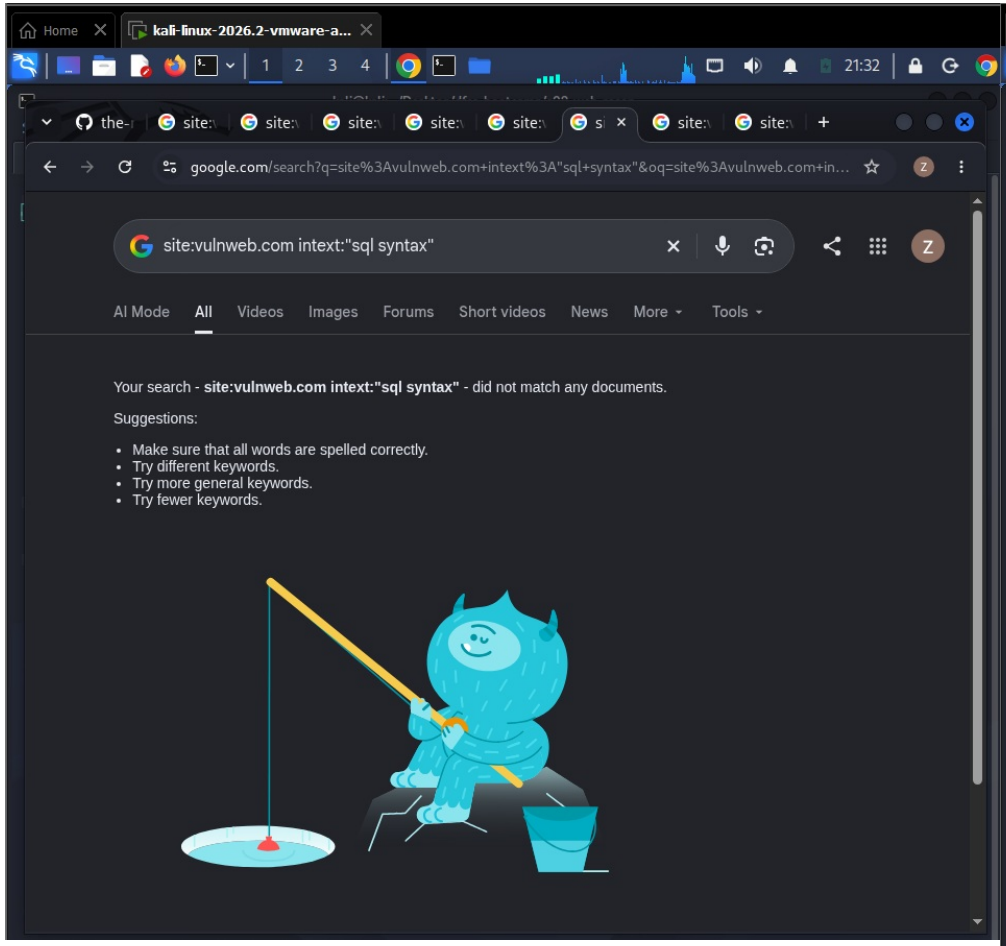
Dork 3: `site:vulnweb.com inurl:admin`



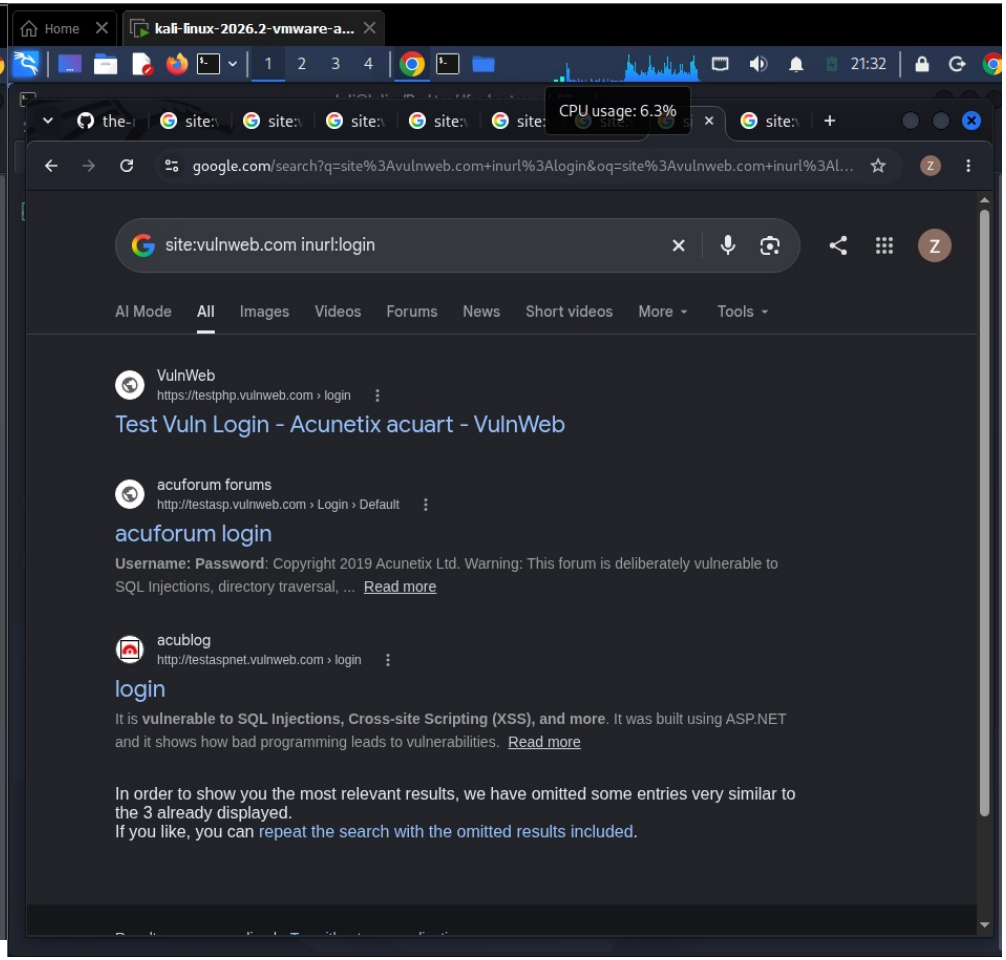
Dork 4: `site:vulnweb.com intitle:"index of"`



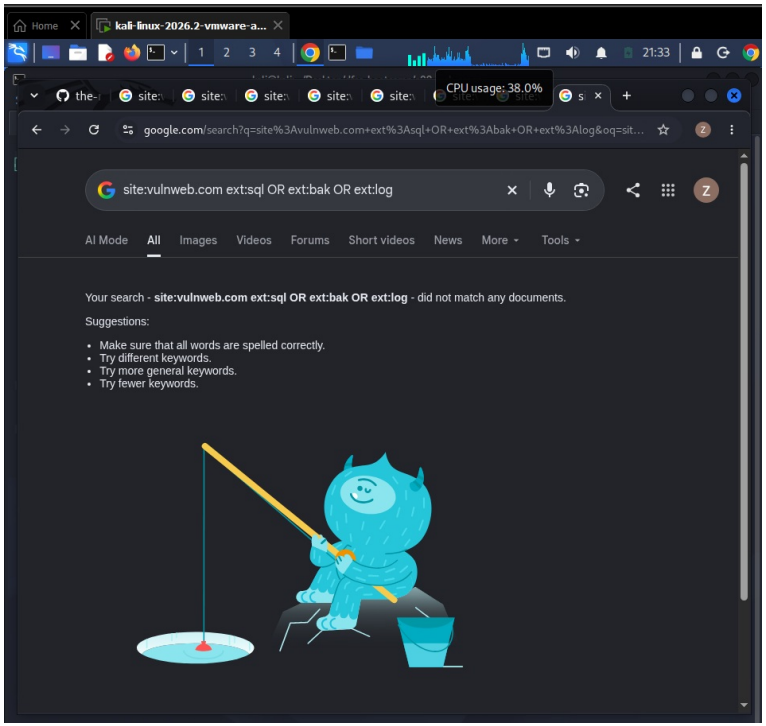
Dork 5: `site:vulnweb.com filetype:php`



Dork 6: `site:vulnweb.com intext:"sql syntax"` – *no results*



Dork 7: `site:vulnweb.com inurl:login`



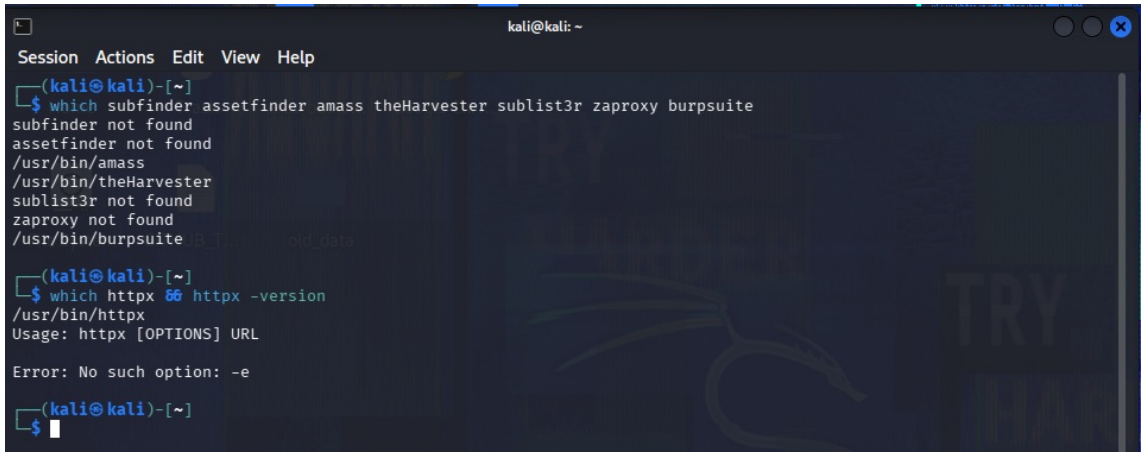
Dork 8: `site:vulnweb.com ext:sql OR ext:bak OR ext:log` – *no results*

Task 3 – Subdomain Reconciliation

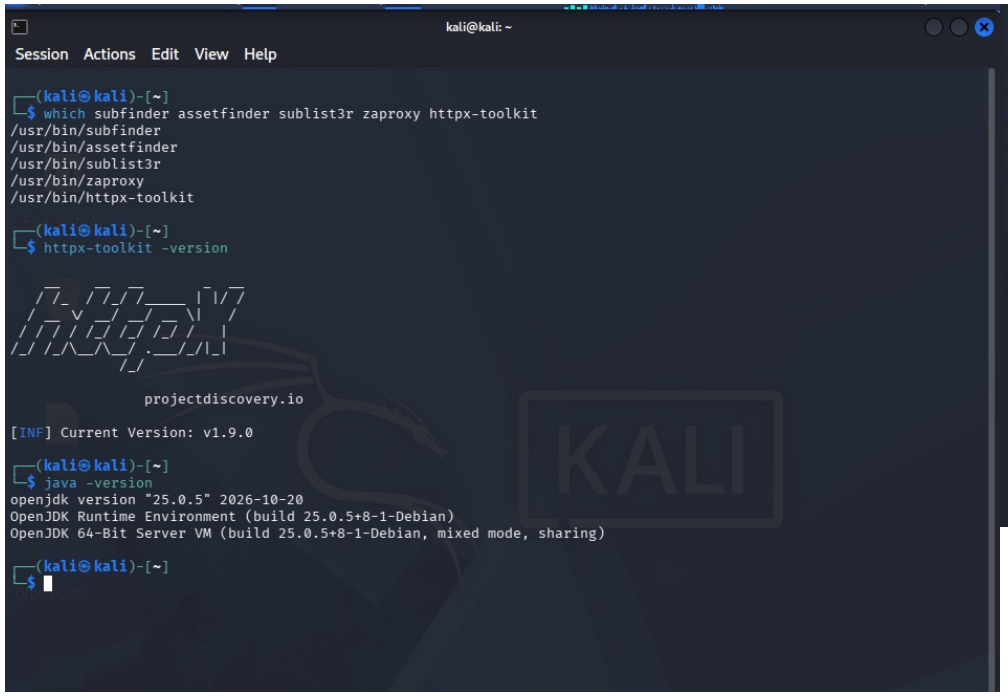
Target: `vulnweb.com`. Enumeration performed passively with three independent tools from Kali.

3.1 – Tool Setup

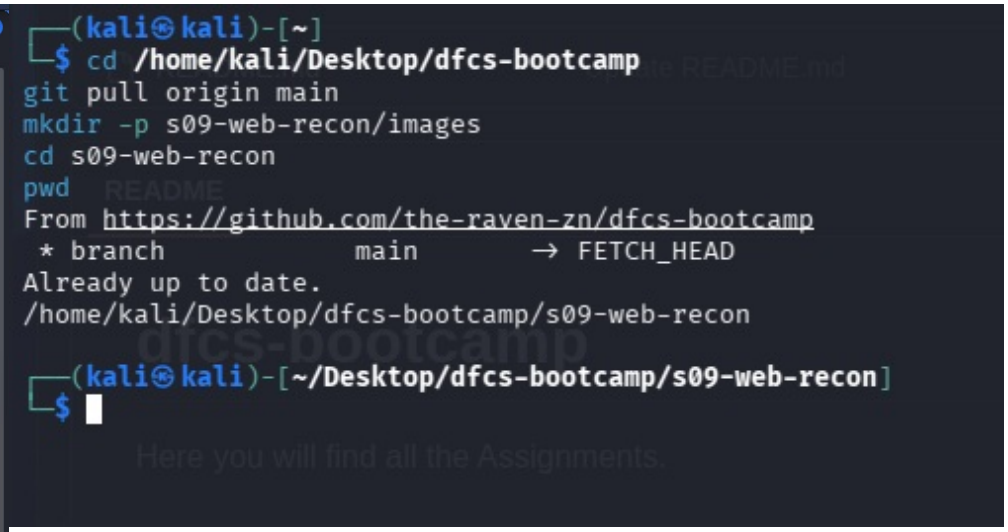
Initial tool check found `subfinder`, `assetfinder`, `sublist3r`, and `zaproxy` missing; `amass`, `theHarvester`, and `burpsuite` were already present. All were installed via `apt`, and `httpx-toolkit` (ProjectDiscovery) was confirmed separately from Kali's conflicting Python `httpx` package.



initial `which` *check*



tools installed, `httpx-toolkit -version`*, Java confirmed*



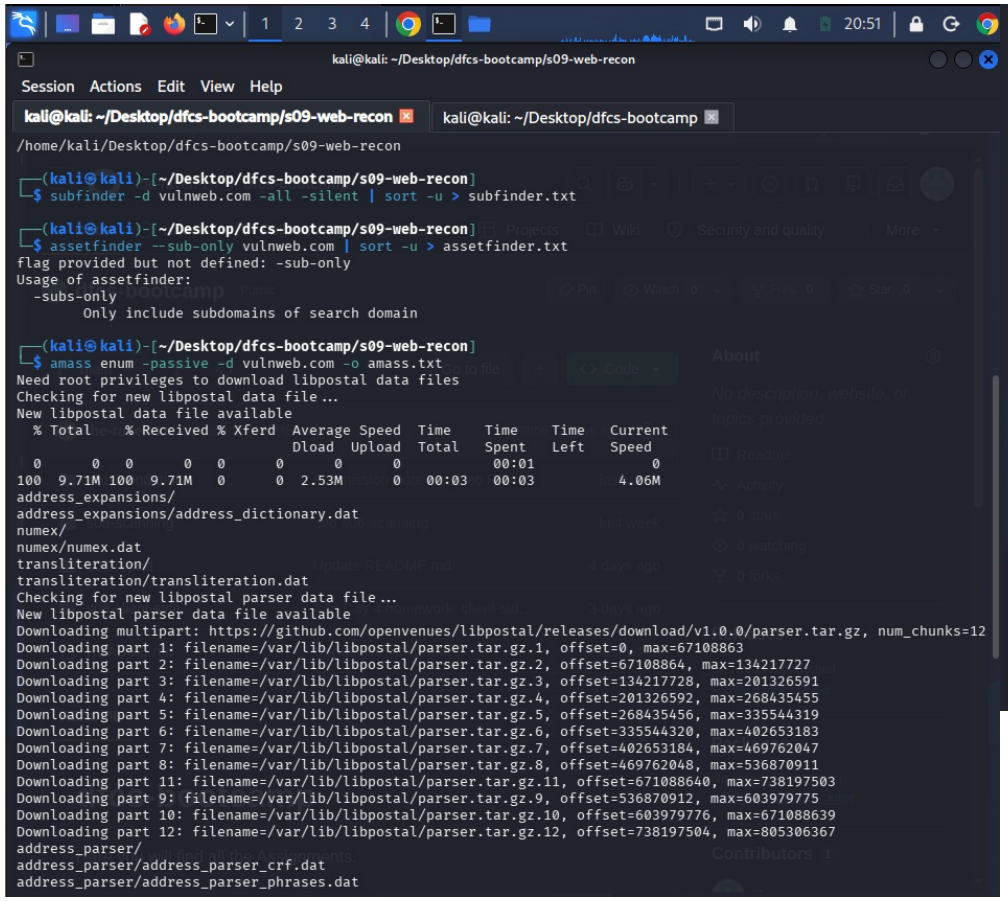
repo pulled, `s09-web-recon/` *folder created*

3.2 – Running the Three Tools

```
subfinder -d vulnweb.com -all -silent | sort -u &gt; subfinder.txt
assetfinder -subs-only vulnweb.com | sort -u &gt; assetfinder.txt
amass enum -passive -d vulnweb.com 2&gt;&amp;1 | tee amass_raw.txt
amass subs -d vulnweb.com -names | sort -u &gt; amass.txt
```

Notes from the run:

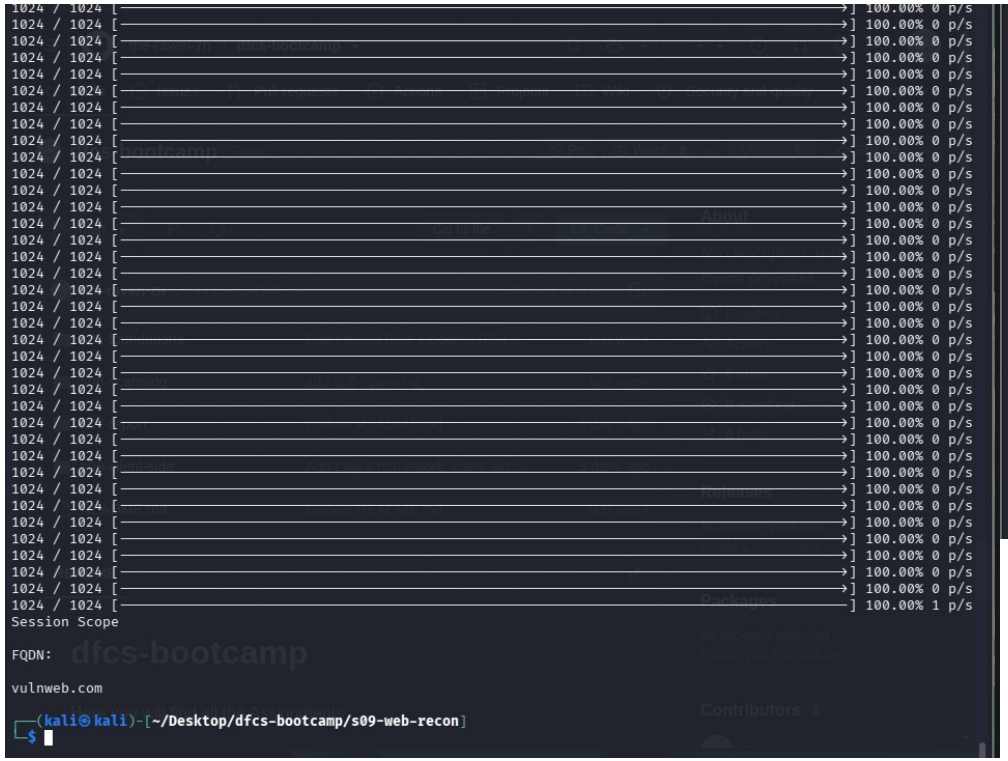
- `assetfinder` on this Kali build takes a single-dash flag (`-subs-only`), not the double-dash form shown in some documentation.
- `amass` v5.1.3 no longer supports `-o` to write results directly; its console output is progress bars only, and the actual hostnames had to be recovered afterwards with `amass subs -d vulnweb.com -names`.
- `sublist3r` was attempted as a possible third/fourth tool but crashed with `TypeError: cannot pickle '_thread.lock' object` on Python 3.14 – a known incompatibility of the older codebase, not a configuration error. It was dropped in favor of `amass`.



amass enum -passive*: libpostal data download (part 1)*



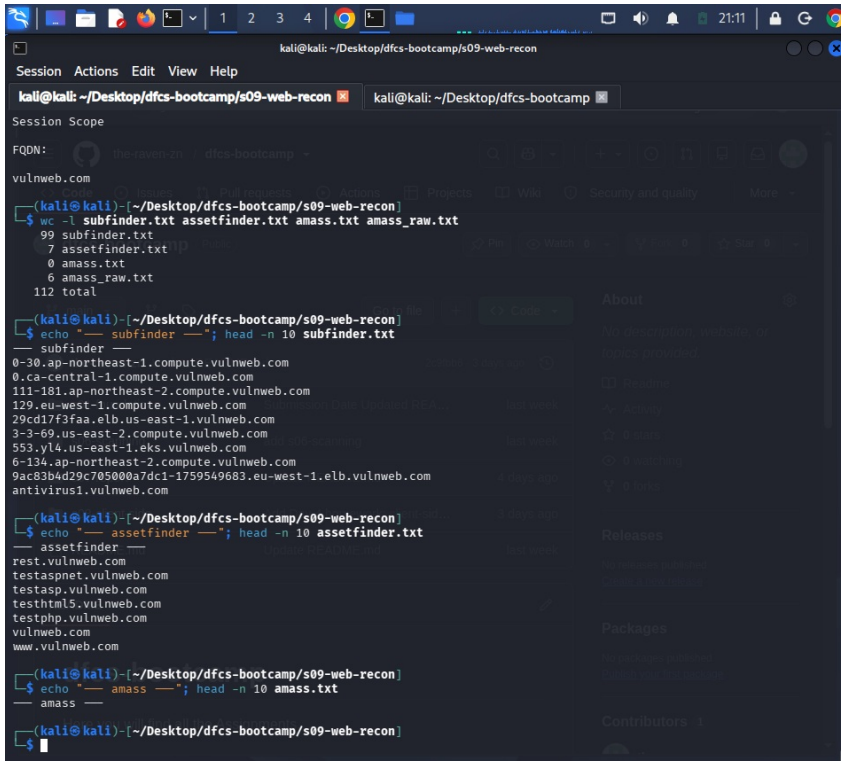
amass enum -passive*: libpostal data download (part 2)*



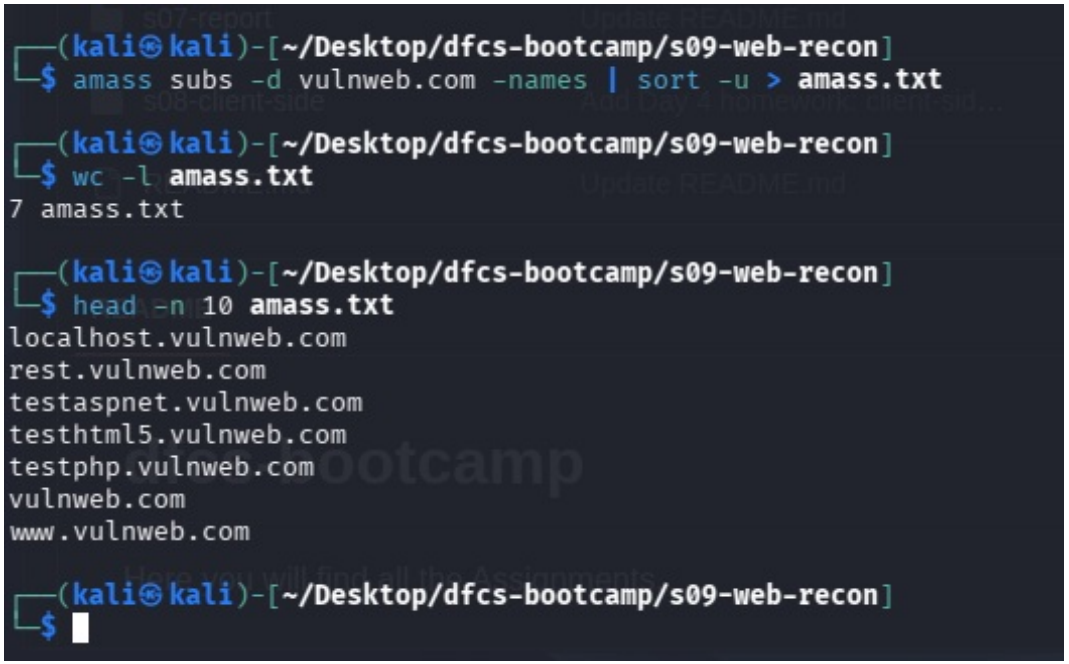
`assetfinder` *run and* `amass enum` *progress bars*



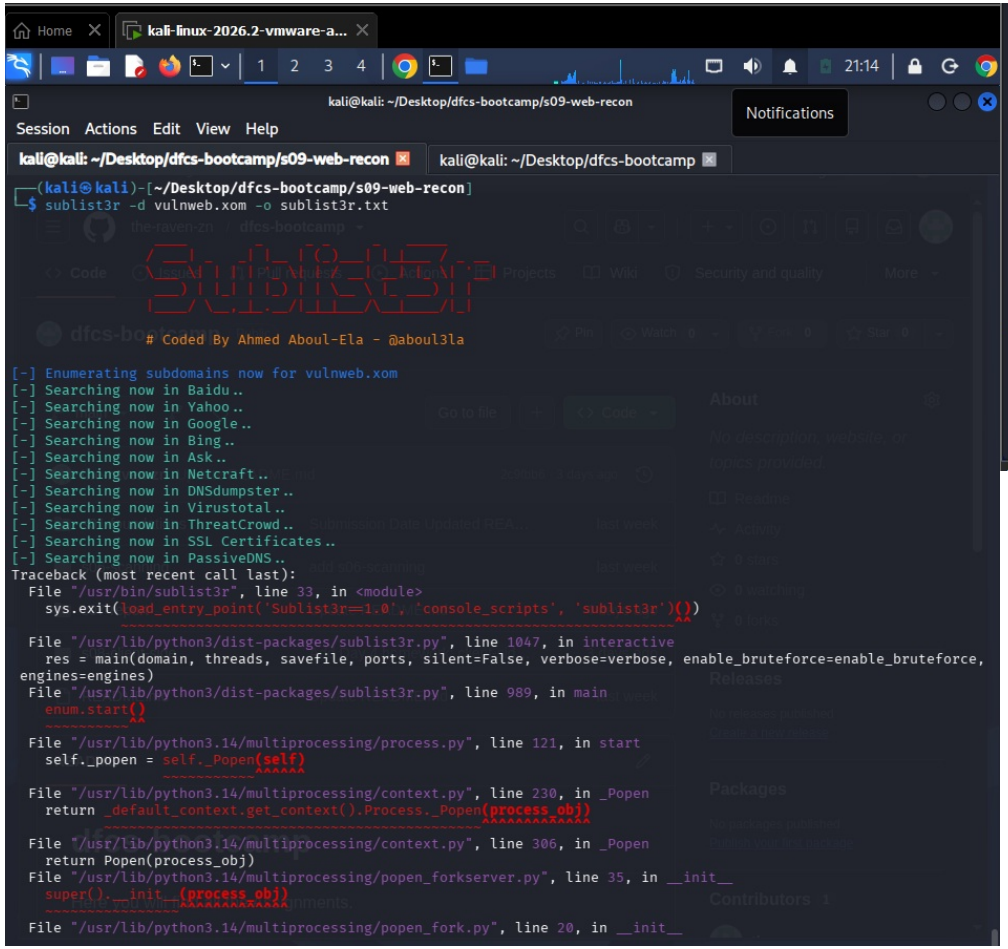
`amass enum` *progress bars continued (passive, no console host output)*



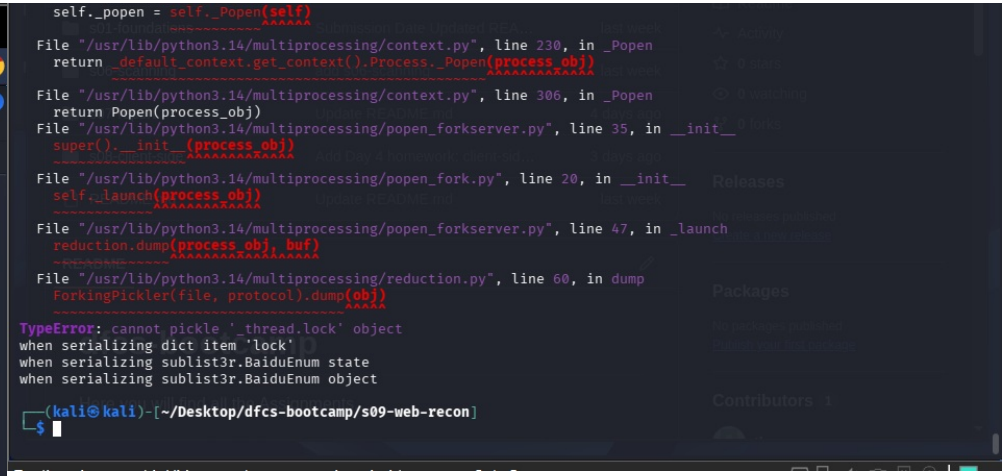
subfinder 99 / assetfinder 7 / amass (raw) 0 – head of each tool's output



`amass subs -d vulnweb.com -names` *recovers 7 real hostnames*



sublist3r attempted as a fourth tool – starts enumerating across 10 search engines



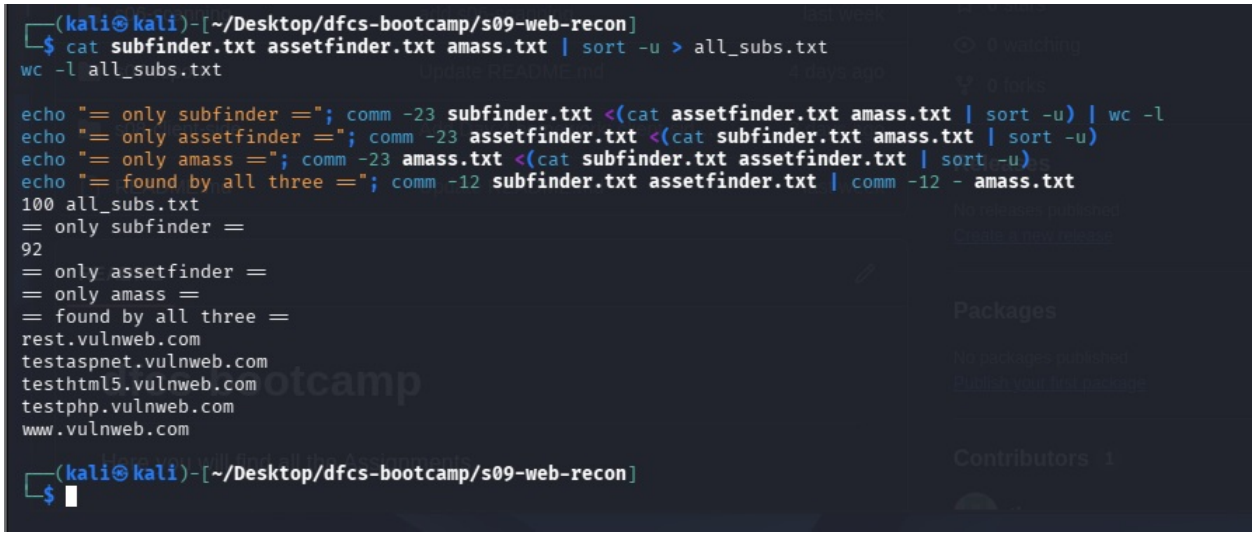
sublist3r crash: `TypeError: cannot pickle '_thread.lock' object` *on Python 3.14*

3.3 – Results Summary

Tool	Hostnames found
subfinder	99
assetfinder	7
amass	7
Union (deduplicated)	**100**

```
cat subfinder.txt assetfinder.txt amass.txt | sort -u &gt; all_subs.txt
comm -23 subfinder.txt &lt;(cat assetfinder.txt amass.txt | sort -u) # only subfinder: 92
comm -23 assetfinder.txt &lt;(cat subfinder.txt amass.txt | sort -u) # only assetfinder: 0
comm -23 amass.txt &lt;(cat subfinder.txt assetfinder.txt | sort -u) # only amass: 0
comm -12 subfinder.txt assetfinder.txt | comm -12 - amass.txt # all three: 5
```

Comparison	Count
Found by all three tools	5 (rest, testaspnet, testhtml5, testphp, www)
Unique to subfinder	92
Unique to assetfinder	0
Unique to amass	0

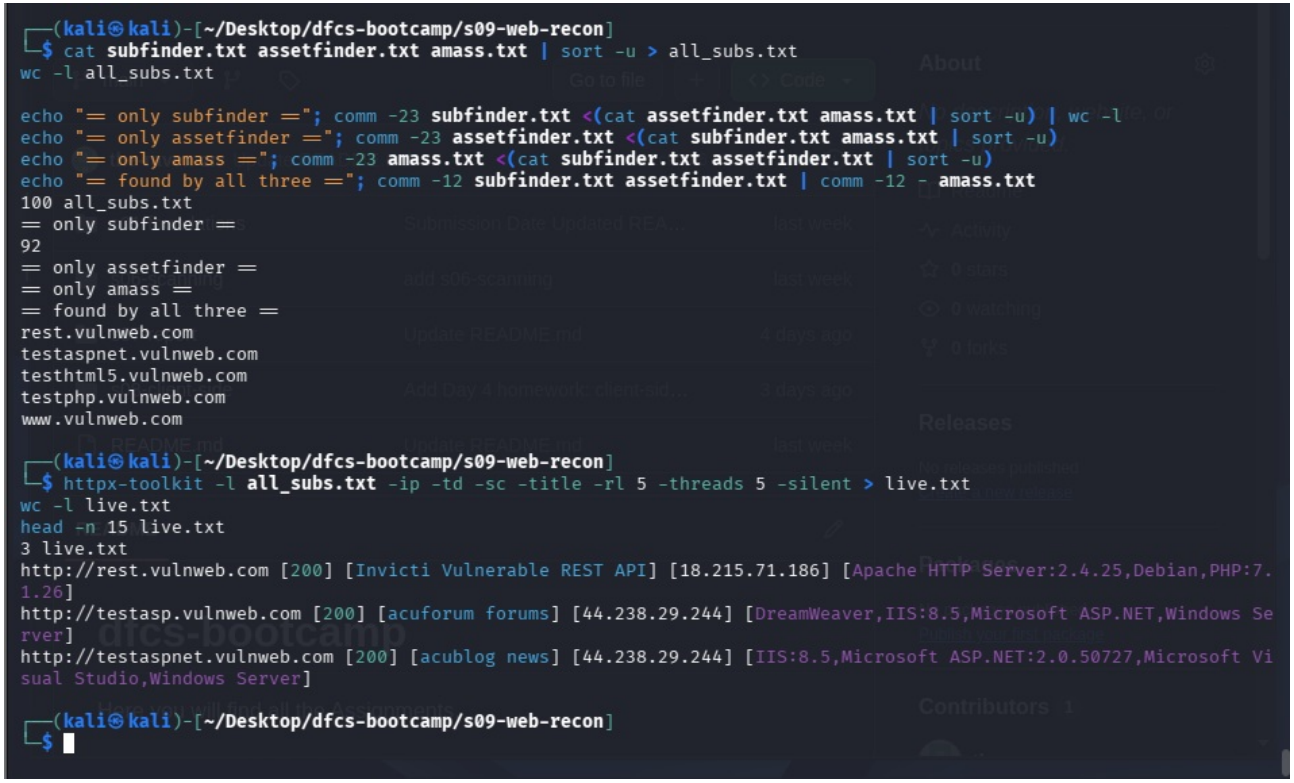


set-comparison commands and output across the three result files

Neither `assetfinder` nor `amass` found anything `subfinder` missed – but both found the bare apex domain `vulnweb.com`, which subfinder's own output did not include (`grep -c "^vulnweb.com$" subfinder.txt` → 0).

3.4 – Resolving the List

```
httpx-toolkit -l all_subs.txt -ip -td -sc -title -rl 5 -threads 5 -silent &gt; live.txt
httpx-toolkit -l core.txt -ip -td -sc -title -timeout 15 -retries 2 -ports 80,443,8080 -silent | tee live_core.txt
while read h; do echo "$h,$(dig +short "$h" | head -n1)"; done &lt; all_subs.txt &gt; resolved.csv
```



all_subs.txt / core.txt resolution via httpx-toolkit; 3 live hosts found

```
┌──(kali㉿kali)-[~/Desktop/dfcs-bootcamp/s09-web-recon]
└─$ printf "rest.vulnweb.com\ntestaspnet.vulnweb.com\ntesthtml5.vulnweb.com\ntestphp.vulnweb.com\nwww.vulnweb.com\nvulnweb.com\n" > core.txt
httpx-toolkit -l core.txt -ip -td -sc -title -timeout 15 -retries 2 -ports 80,443,8080 -silent | tee live_core.txt
http://rest.vulnweb.com [200] [Invicti Vulnerable REST API] [18.215.71.186] [Apache HTTP Server:2.4.25,Debian,PHP:7.1.26]
http://testaspnet.vulnweb.com [200] [acublog news] [44.238.29.244] [IIS:8.5,Microsoft ASP.NET:2.0.50727,Microsoft Visual Studio,Windows Server]
http://testaspnet.vulnweb.com [200] [acublog news] [44.238.29.244] [IIS:8.5,Microsoft ASP.NET:2.0.50727,Microsoft Visual Studio,Windows Server]
http://rest.vulnweb.com [200] [Invicti Vulnerable REST API] [18.215.71.186] [Apache HTTP Server:2.4.25,Debian,PHP:7.1.26]

┌──(kali㉿kali)-[~/Desktop/dfcs-bootcamp/s09-web-recon]
└─$ dig +short thisdoesnotexist12345.compute.vulnweb.com
dig +short 0-30.ap-northeast-1.compute.vulnweb.com
dig +short vulnweb.com
grep -c "^vulnweb.com$" subfinder.txt
44.228.249.3
0

┌──(kali㉿kali)-[~/Desktop/dfcs-bootcamp/s09-web-recon]
└─$ grep -c "compute" subfinder.txt
15

┌──(kali㉿kali)-[~/Desktop/dfcs-bootcamp/s09-web-recon]
└─$
```

resolving / non-resolving split, and a sample of the non-compute, non-resolving names

```
┌──(kali㉿kali)-[~/Desktop/dfcs-bootcamp/s09-web-recon]
└─$ while read h; do echo "$h,$(dig +short "$h" | head -n1)"; done < all_subs.txt > resolved.csv
echo "resolving:"; awk -F, '$2≠""' resolved.csv | wc -l
echo "not resolving:"; awk -F, '$2==""' resolved.csv | wc -l
echo "── sample of non-compute, non-resolving names ──"
grep -vE "compute" subfinder.txt | head -n 20
resolving:
8
not resolving:
92
── sample of non-compute, non-resolving names ──
29cd17f3faa.elb.us-east-1.vulnweb.com
553.yl4.us-east-1.eks.vulnweb.com
9ac83b4d29c705000a7dc1-1759549683.eu-west-1.elb.vulnweb.com
antivirus1.vulnweb.com
autoconfig.vulnweb.com
b0-p5rxx4le-eu-north-1c.s3.eu-north-1.vpce.vulnweb.com
baidu.vulnweb.com
bing.com.vulnweb.com
blog.vulnweb.com
class.vulnweb.com
cn.vulnweb.com
content.vulnweb.com
cs.vulnweb.com
dev.vulnweb.com
digital-defense-load-balancer-1314805317.us-east-1.elb.vulnweb.com
esthtml5.vulnweb.com
estphp.vulnweb.com
gd3.vulnweb.com
gd.vulnweb.com
groups.vulnweb.com

┌──(kali㉿kali)-[~/Desktop/dfcs-bootcamp/s09-web-recon]
└─$ curl -sI -m 10 http://testphp.vulnweb.com | head -n 3
^[[A

┌──(kali㉿kali)-[~/Desktop/dfcs-bootcamp/s09-web-recon]
└─$ curl -sI -m 10 http://testphp.vulnweb.com | head -n 3
```

curl timing tests on testphp.vulnweb.com (timeout, exit 28)

```
┌──(kali㉿kali)-[~/Desktop/dfcs-bootcamp/s09-web-recon]
└─$ awk -F, '$2≠""' resolved.csv
curl -sI -m 10 http://testphp.vulnweb.com | head -n 3; echo "curl exit: ${PIPESTATUS[0]}"
localhost.vulnweb.com,127.0.0.1
rest.vulnweb.com,18.215.71.186
testaspnet.vulnweb.com,44.238.29.244
testasp.vulnweb.com,44.238.29.244
testhtml5.vulnweb.com,44.228.249.3
testphp.vulnweb.com,44.228.249.3
vulnweb.com,44.228.249.3
www.vulnweb.com,44.228.249.3
curl exit:

┌──(kali㉿kali)-[~/Desktop/dfcs-bootcamp/s09-web-recon]
└─$
```

resolved.csv: 4 names share one IP (44.228.249.3), all returning connection timeouts

```
┌──(kali㉿kali)-[~/Desktop/dfcs-bootcamp/s09-web-recon]
└─$ curl -sI -m 10 http://testphp.vulnweb.com; echo "curl exit: $?"
curl exit: 28

┌──(kali㉿kali)-[~/Desktop/dfcs-bootcamp/s09-web-recon]
└─$
```

control check – `curl exit: 28` confirms timeout, not a wildcard DNS artifact

Stage	Count
Combined unique names	100
Resolve in DNS	8
Respond over HTTP (200)	3 (rest, testasp, testaspnet)

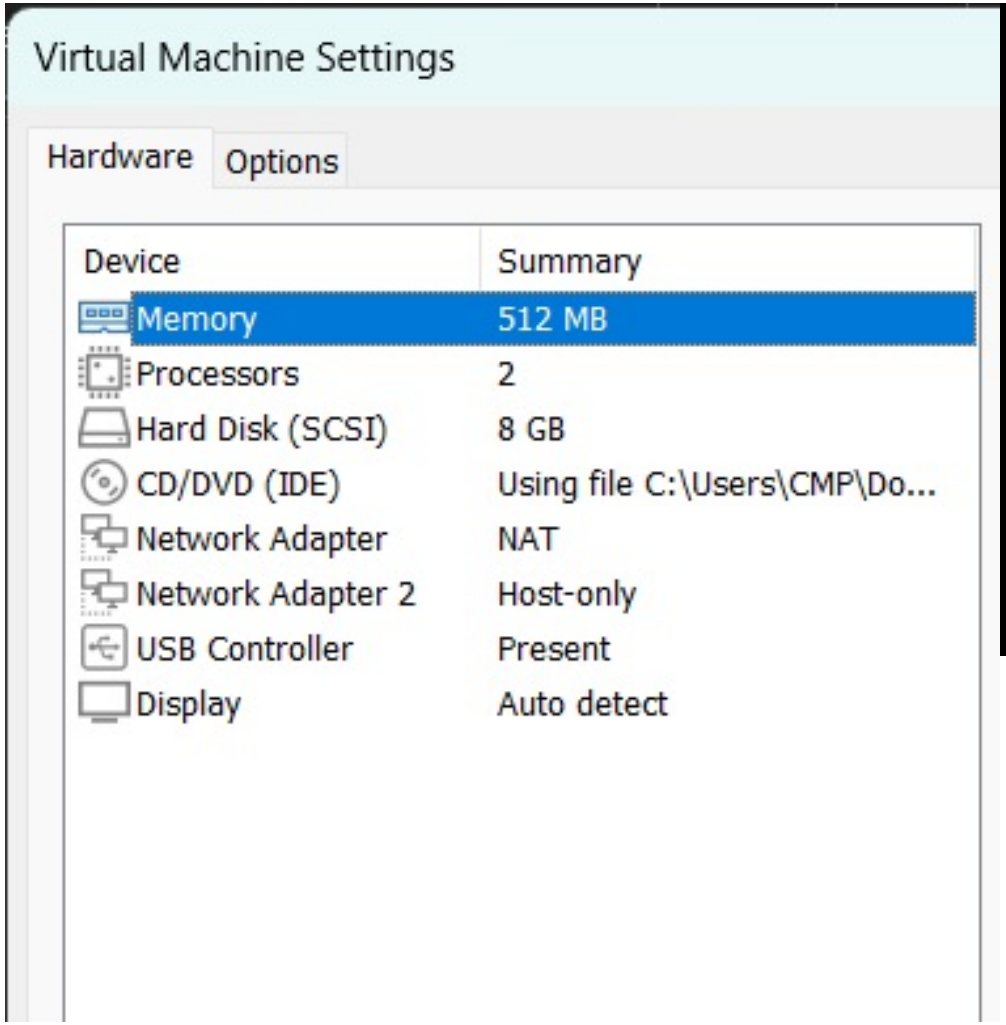
Of the 8 resolving names, 4 (`testhtml5`, `testphp`, `vulnweb.com`, `www`) share one IP (`44.228.249.3`) and all returned a connection timeout (curl exit code 28 rather than a response) – one shared server issue, not four separate findings. `localhost.vulnweb.com` resolves to the loopback address `127.0.0.1`, a DNS misconfiguration worth noting even though it cannot be tested directly. A control check with a name that cannot exist (`thisdoesnotexist12345.compute.vulnweb.com`) returned nothing, ruling out a wildcard DNS record as the explanation for the many cloud-style names.

3.5 – Conclusion

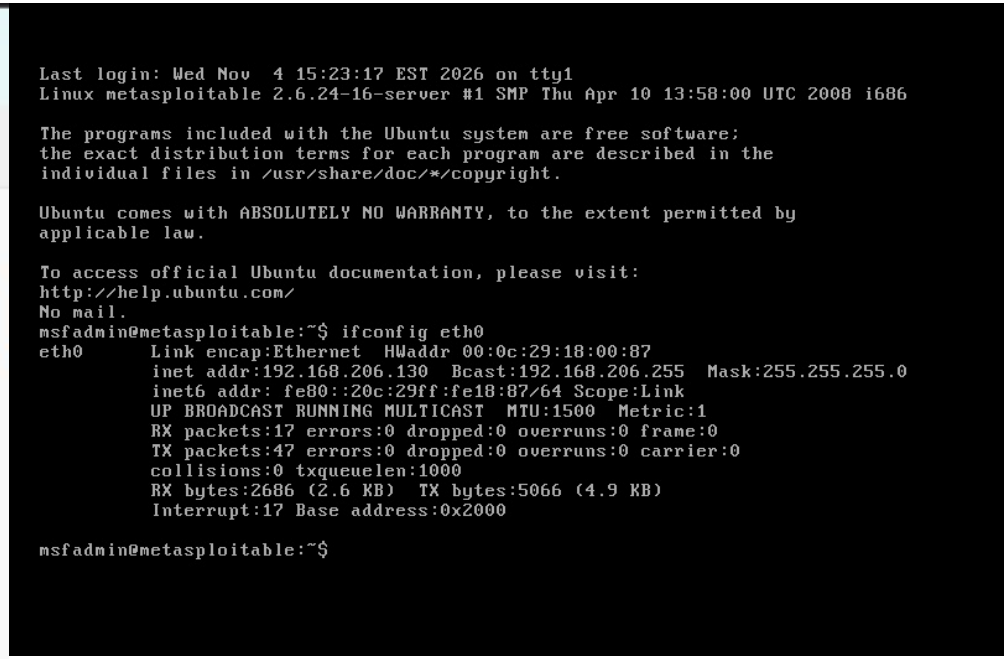
Subfinder returned 99 names, 92 of them found by no other tool, because it draws on certificate transparency logs and passive DNS datasets, which include many historical or machine-generated names (such as auto-scaling cloud load-balancer hostnames). Assetfinder and amass each returned 7 names that overlapped almost completely with each other and with subfinder's core results. Only 5 names were found by all three tools. Neither assetfinder nor amass found a name that subfinder had missed, but both found the bare domain `vulnweb.com`, which subfinder did not report – a concrete example of a simpler tool catching something a larger one missed. After DNS resolution, only 8 of the 100 combined names resolved, and only 3 responded as live web servers. This demonstrates why the union of several tools is larger than any single tool, and why resolution is mandatory before a subdomain list is used: most passive-source results are historical noise rather than live infrastructure.

Task 4 – Scanner Comparison: ZAP vs Burp Suite

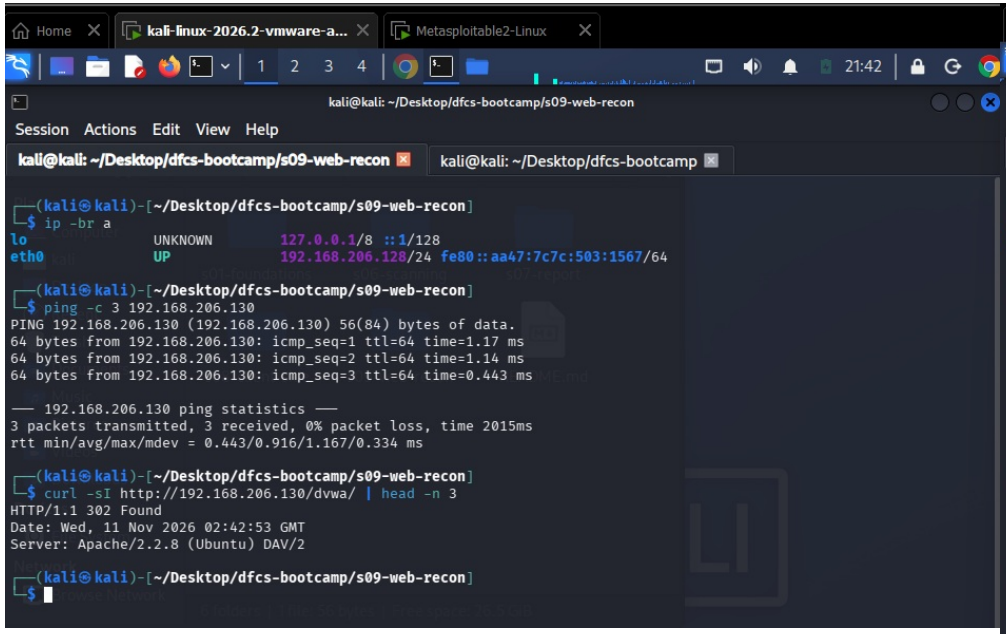
Target: Metasploitable 2 (`192.168.206.130`) running DVWA, Security Level: Low. **Attacking machine:** Kali (`192.168.206.128`), same virtual network, isolated from the internet during testing.



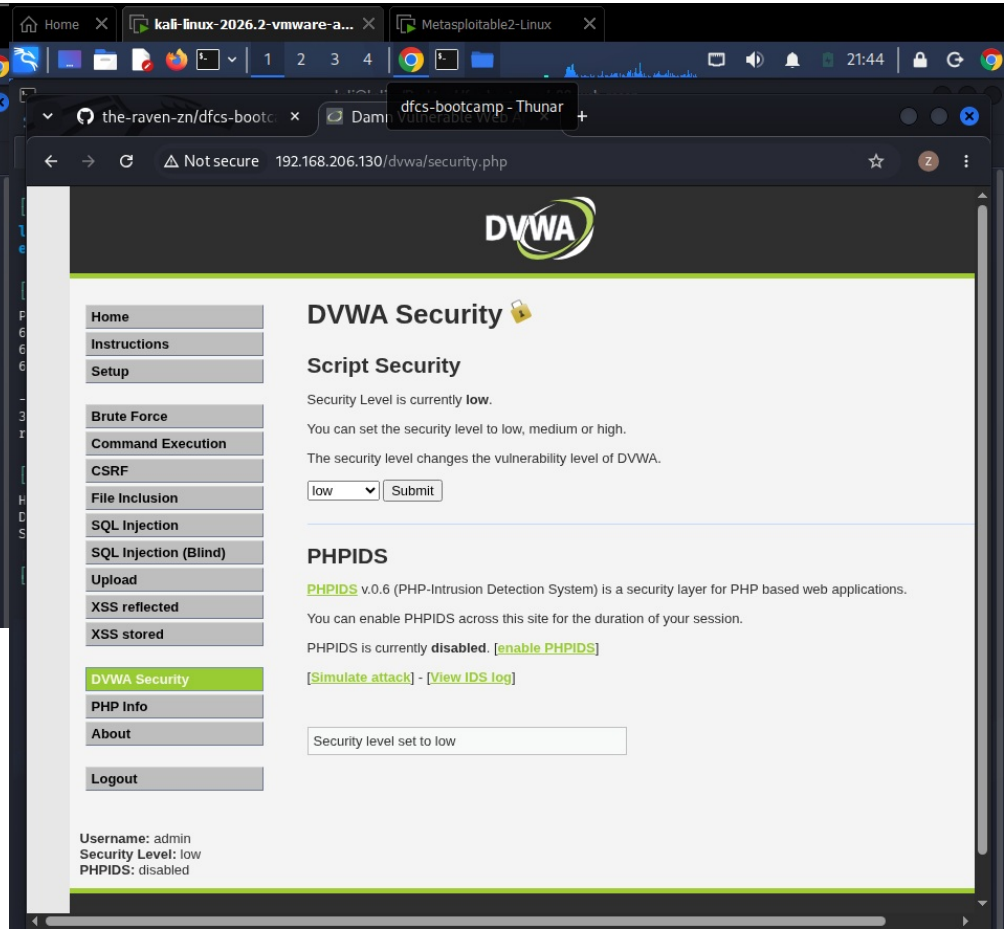
VM settings: 512 MB RAM, 2 vCPU, host-only + NAT adapters



Metasploitable 2 boot and `ifconfig eth0` confirming 192.168.206.130



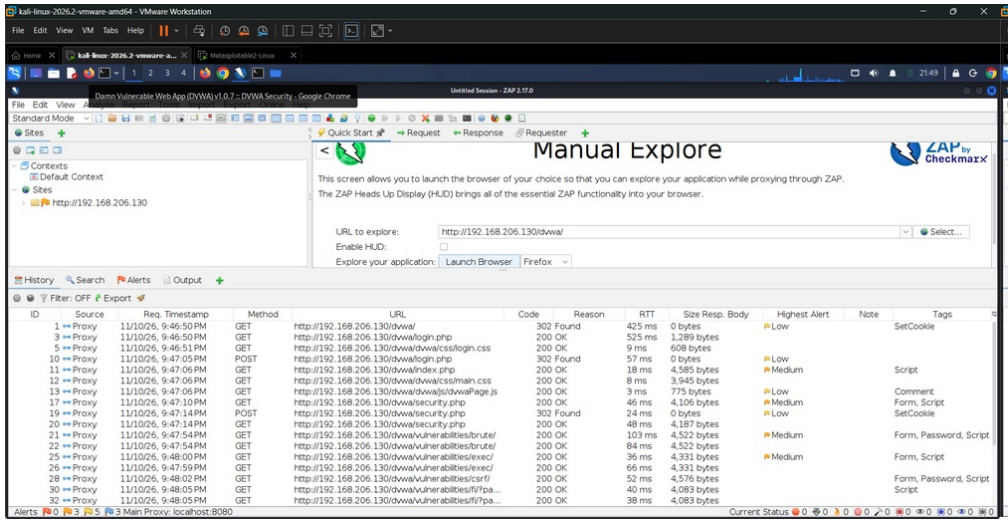
connectivity check: `ping` *and* `curl` *to /dvwa/ from Kali*



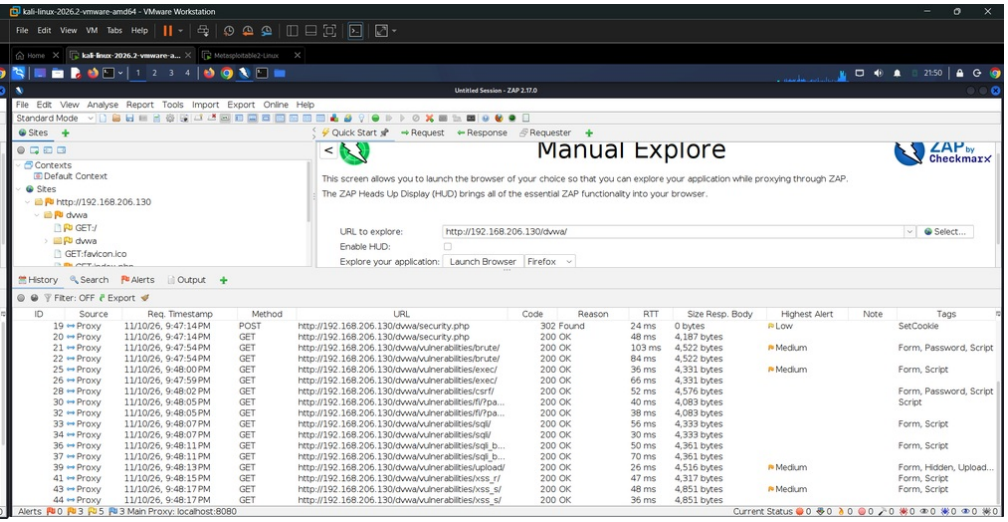
DVWA Security page: Script Security set to Low, PHPIDS disabled

4.1 – OWASP ZAP

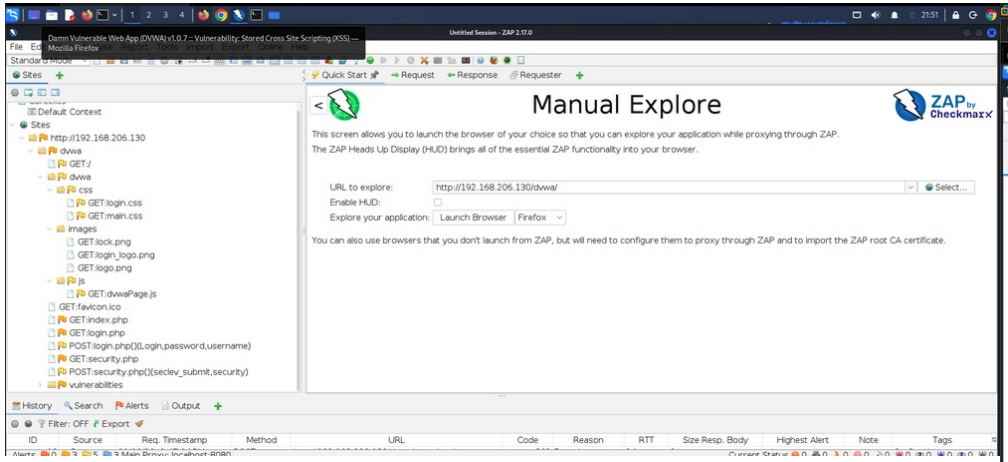
Method: Manual Explore (browsing every DVWA vulnerability page through ZAP's proxy to populate the site tree), followed by a full Active Scan.



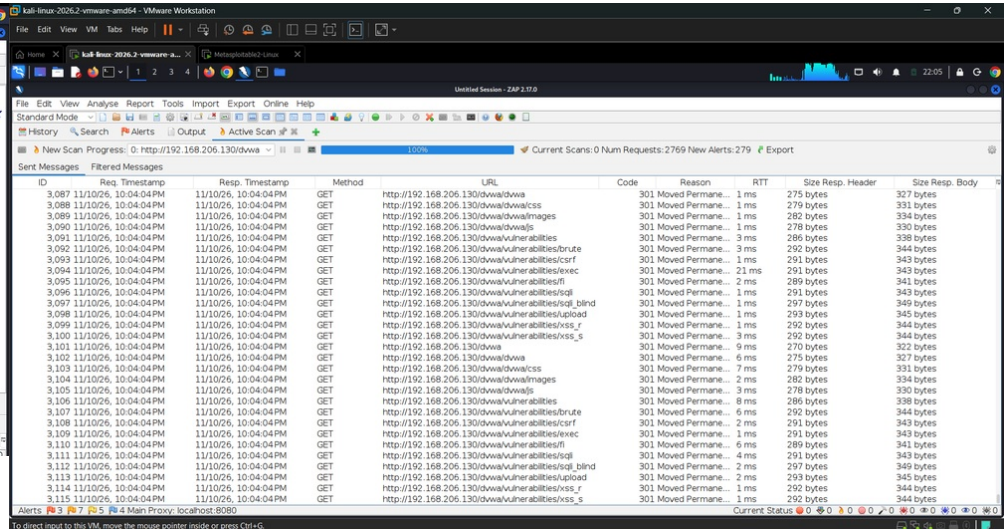
Manual Explore: proxy history after browsing DVWA's vulnerability pages (part 1)



Manual Explore: proxy history continued (part 2)



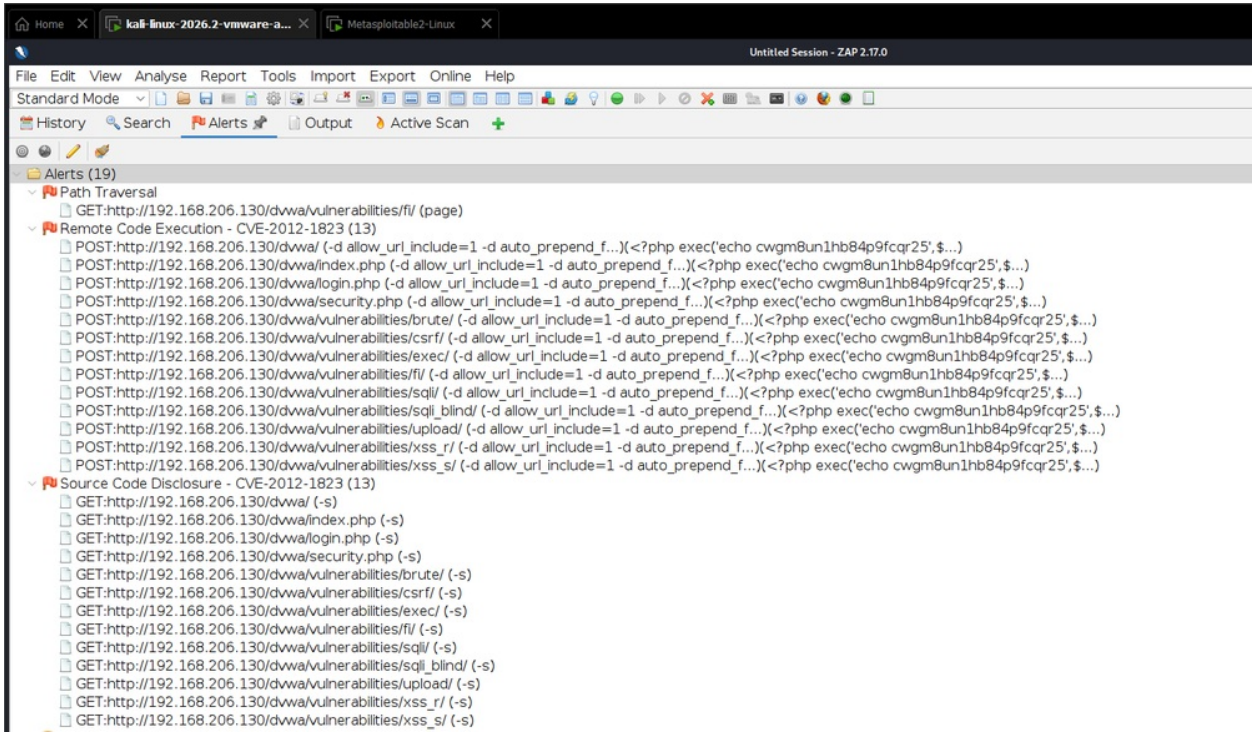
Manual Explore: site tree populated under `/dvwa/vulnerabilities/`



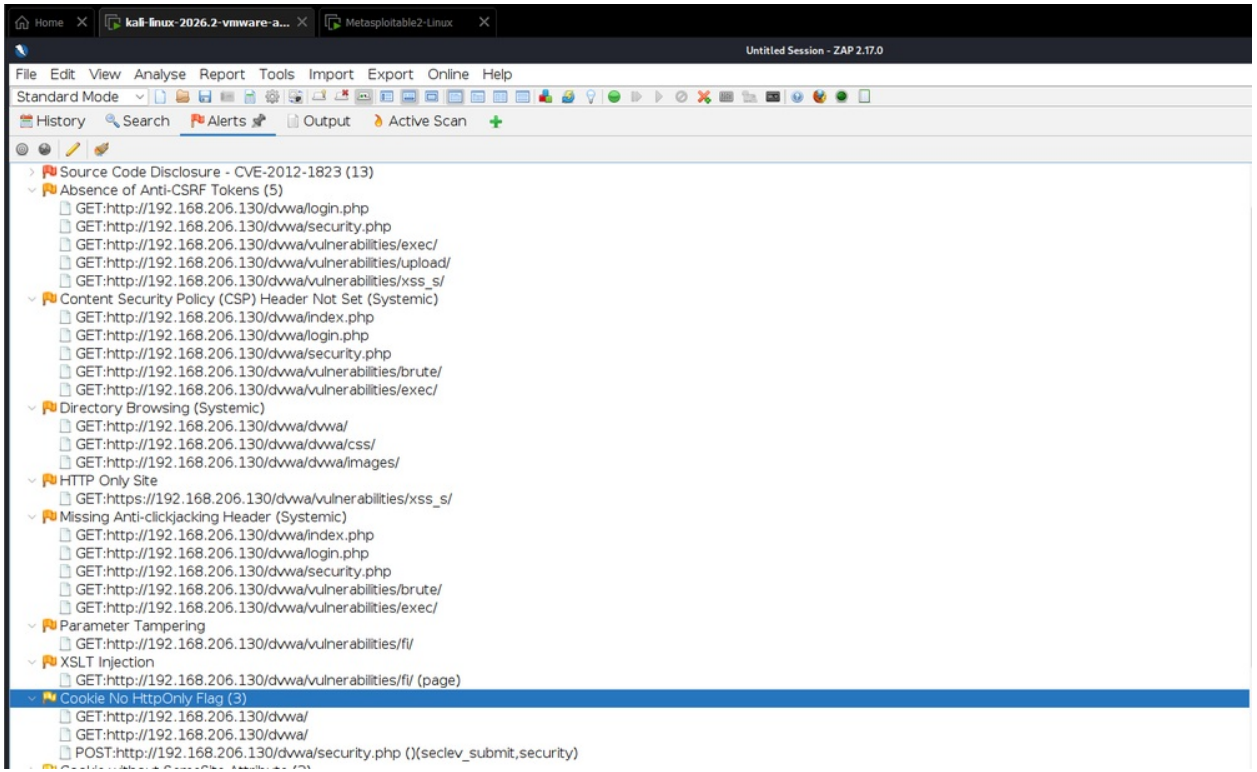
Active Scan complete: 2,769 requests sent

Alerts – 19 distinct findings, 279 instances:

Risk	Count	Key findings
High	3	Path Traversal (`fi` page); Remote Code Execution – CVE-2012-1823 (13 endpoints); Source Code Disclosure – CVE-2012-1823 (13 endpoints)
Medium	7	Absence of Anti-CSRF Tokens; CSP Header Not Set; Directory Browsing; HTTP Only Site; Missing Anti-clickjacking Header; Parameter Tampering; XSLT Injection
Low	5	Cookie No HttpOnly Flag; Cookie without SameSite Attribute; Server Leaks “X-Powered-By”; Server Leaks Version via “Server” header; X-Content-Type-Options Missing
Informational	4	Authentication Request Identified; Cookie Poisoning; Session Management Response Identified; User Agent Fuzzer



ZAP Alerts tree: High-risk Path Traversal, RCE (CVE-2012-1823) and Source Code Disclosure



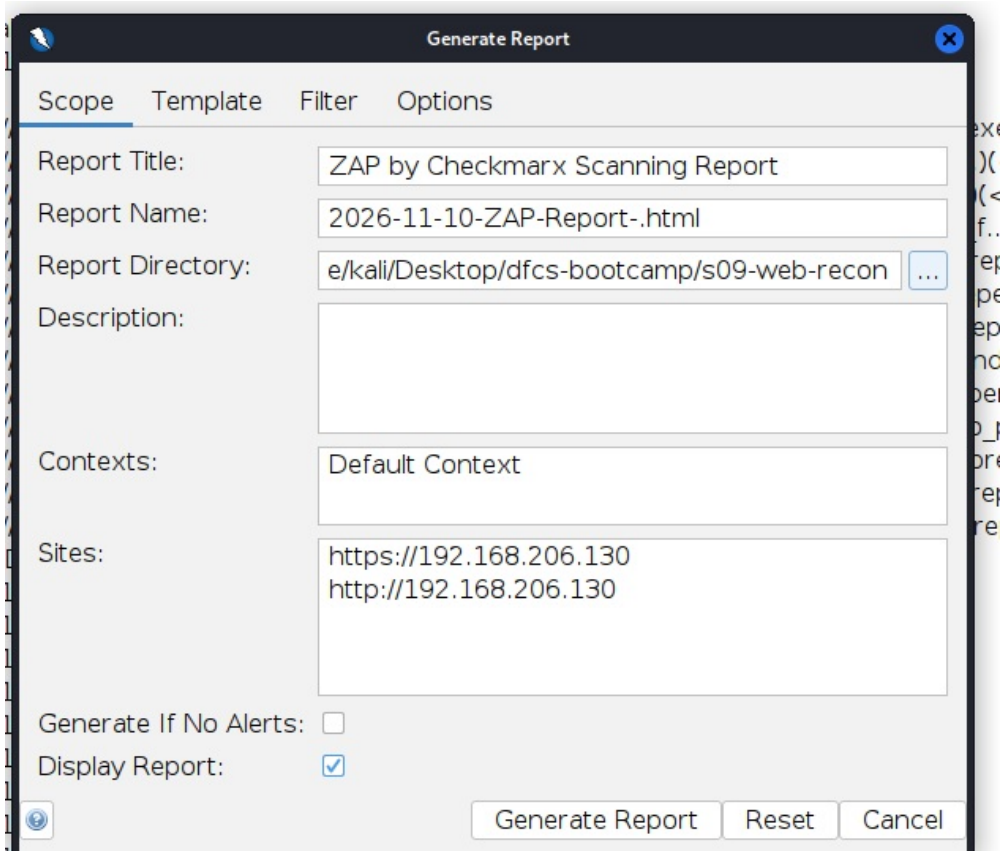
ZAP Alerts tree continued: Medium/Low findings – CSRF, CSP, directory browsing, cookie flags, header leaks



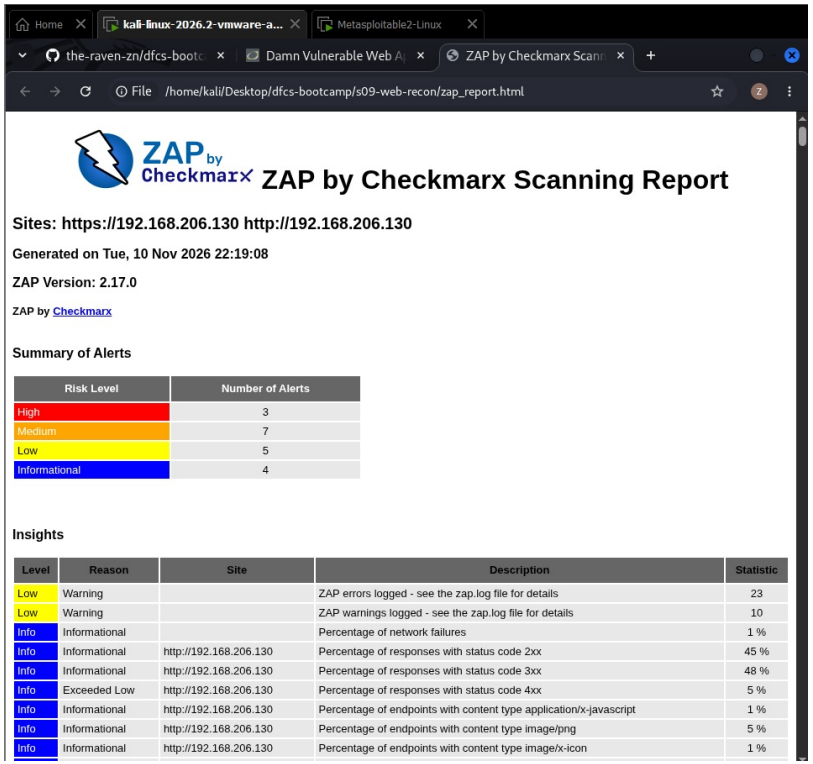
ZAP Alerts tree: Informational findings – authentication request, cookie poisoning, session management, user agent fuzzer

- Authentication Request Identified
  - POST:http://192.168.206.130/dvwa/login.php ()(Login,password,username)
- Cookie Poisoning
  - POST:http://192.168.206.130/dvwa/security.php ()(seclev_submit,security)
- Session Management Response Identified (2)
  - GET:http://192.168.206.130/dvwa/
  - GET:http://192.168.206.130/dvwa/
- User Agent Fuzzer (Systemic)
  - GET:http://192.168.206.130/dvwa/
  - GET:http://192.168.206.130/dvwa/
  - GET:http://192.168.206.130/dvwa/dvwa/css

Low/Informational alert detail (continued)



Generate Report dialog: Traditional HTML Report, scope = 192.168.206.130

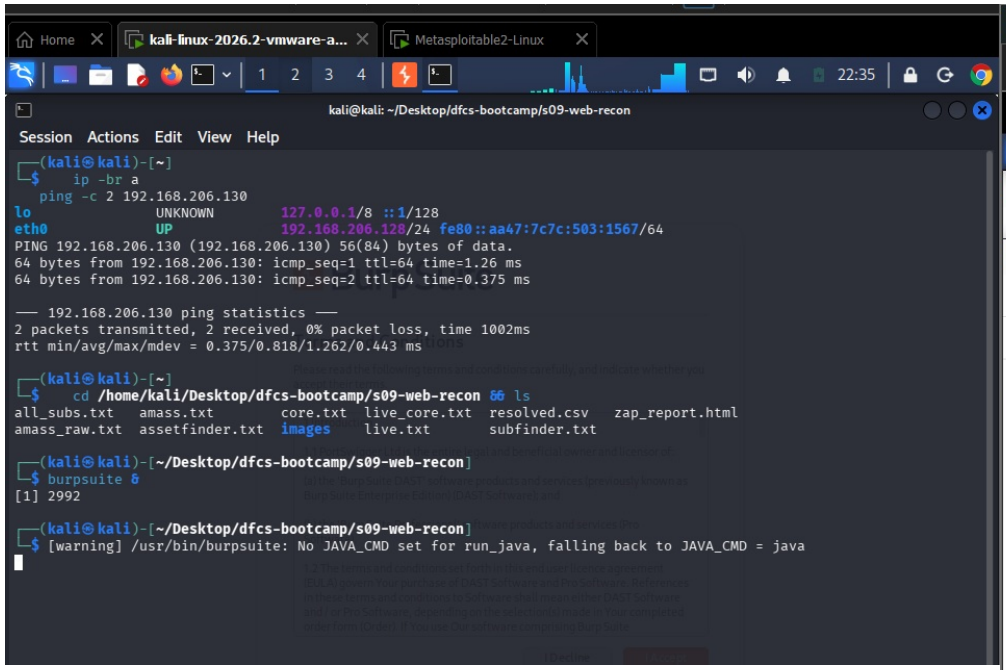


zap_report.html – Traditional HTML Report: 3 High, 7 Medium, 5 Low, 4 Informational

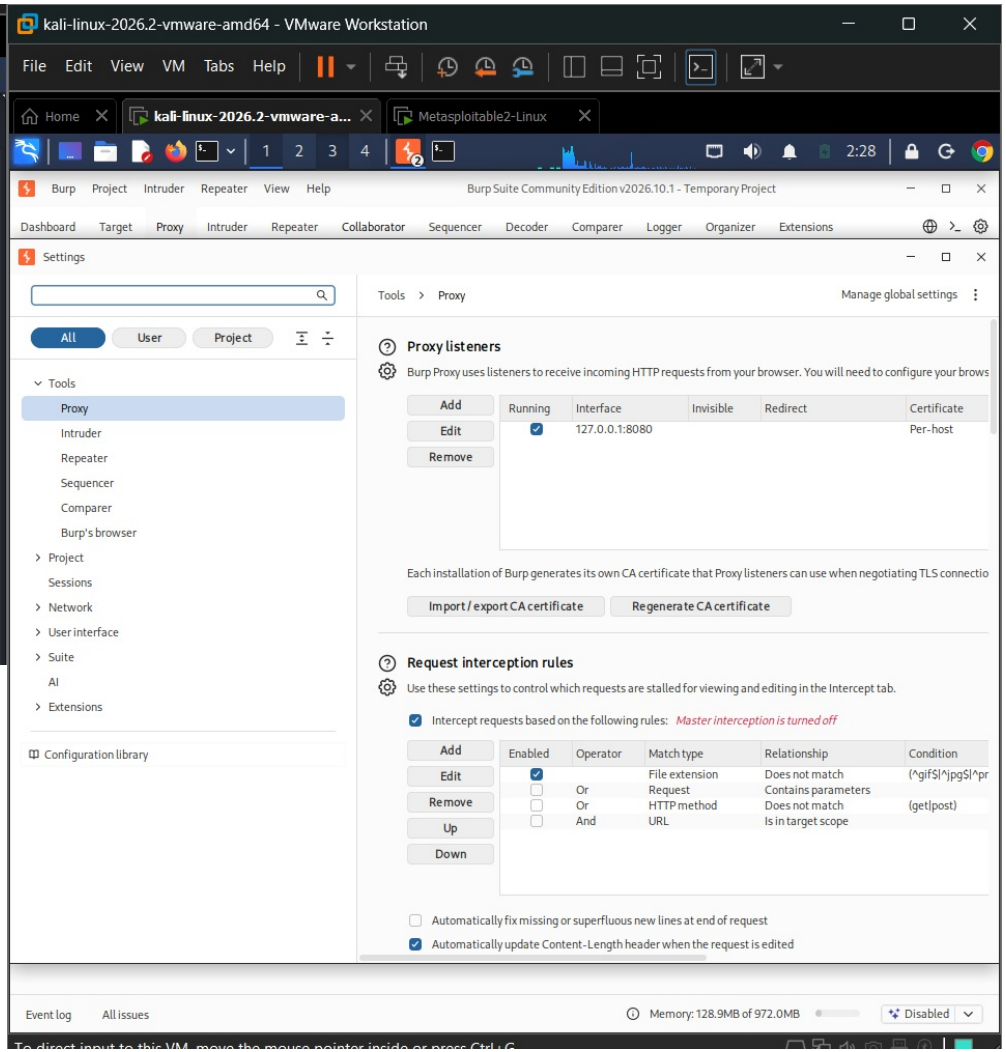
The two High-risk CVE-2012-1823 alerts (RCE and Source Code Disclosure) share one root cause – an old PHP-CGI argument-injection flaw on Metasploitable's outdated PHP – appearing across 13 URLs each because the same vulnerable request pattern applies to every page, not 26 independent bugs. The ZAP report's own Insights table notes that **48% of all responses during the scan were 3xx redirects**, which explains both the scan's long duration and why path/RCE-style findings dominated: ZAP's crawl followed redirects extensively but did not submit the SQL Injection or XSS forms with attacker-chosen values during the automated phase.

4.2 – Burp Suite Community Edition

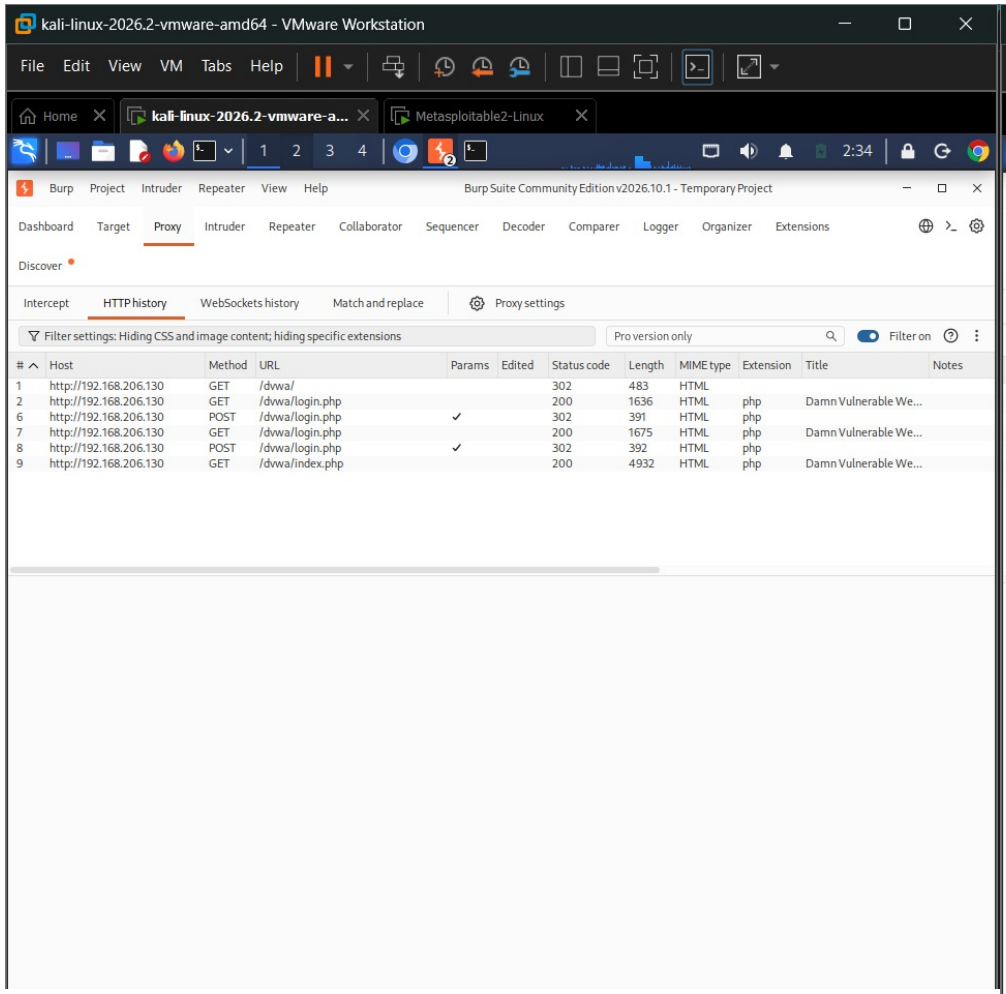
Method: Proxy + manual browsing (Community's automated Crawl and Audit is disabled), followed by manual exploitation in Repeater.



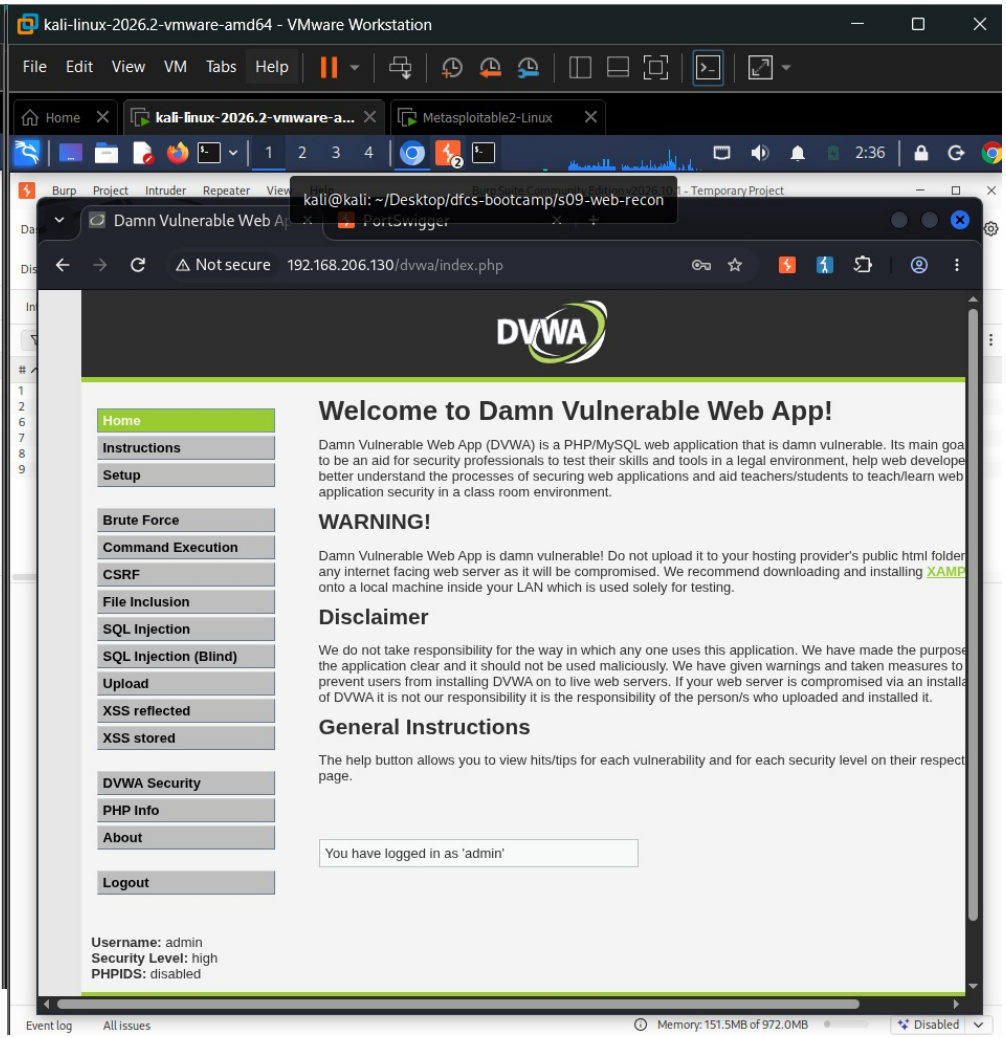
`burpsuite` *launch;* `JAVA_CMD` *warning, falls back to* `java`



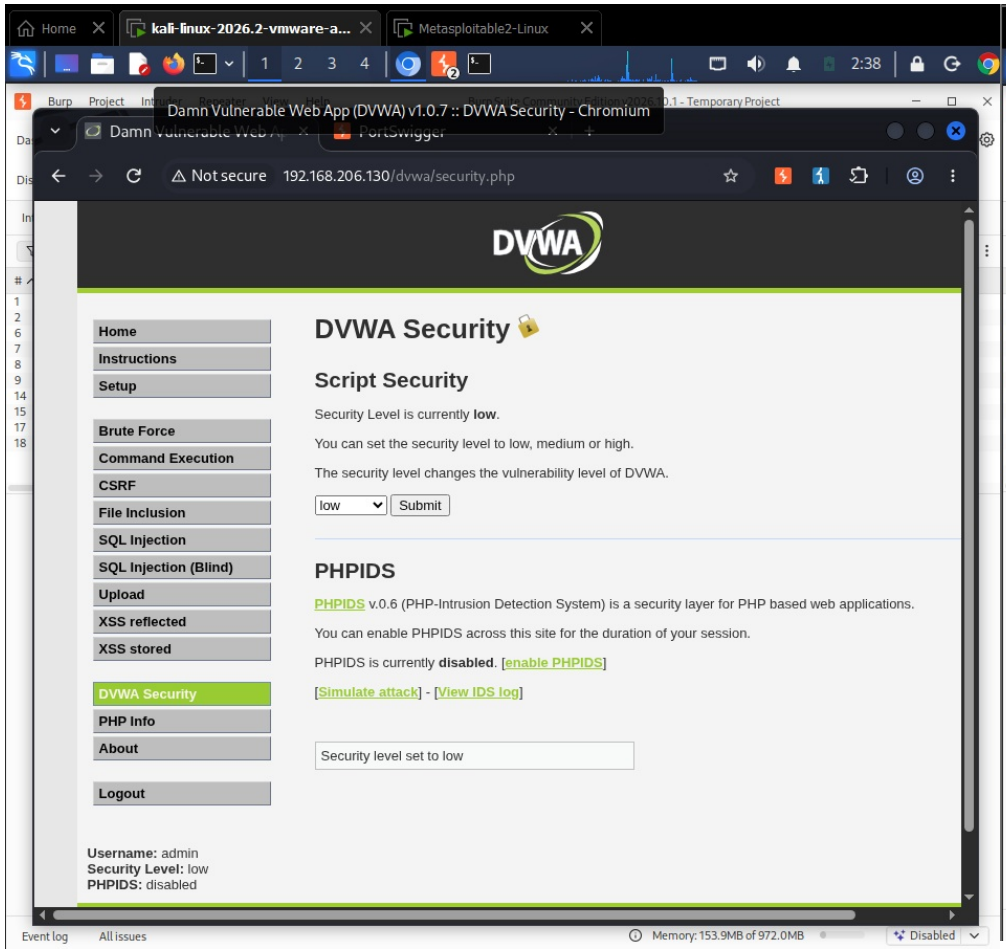
Burp proxy listener on 127.0.0.1:8080 and interception rules



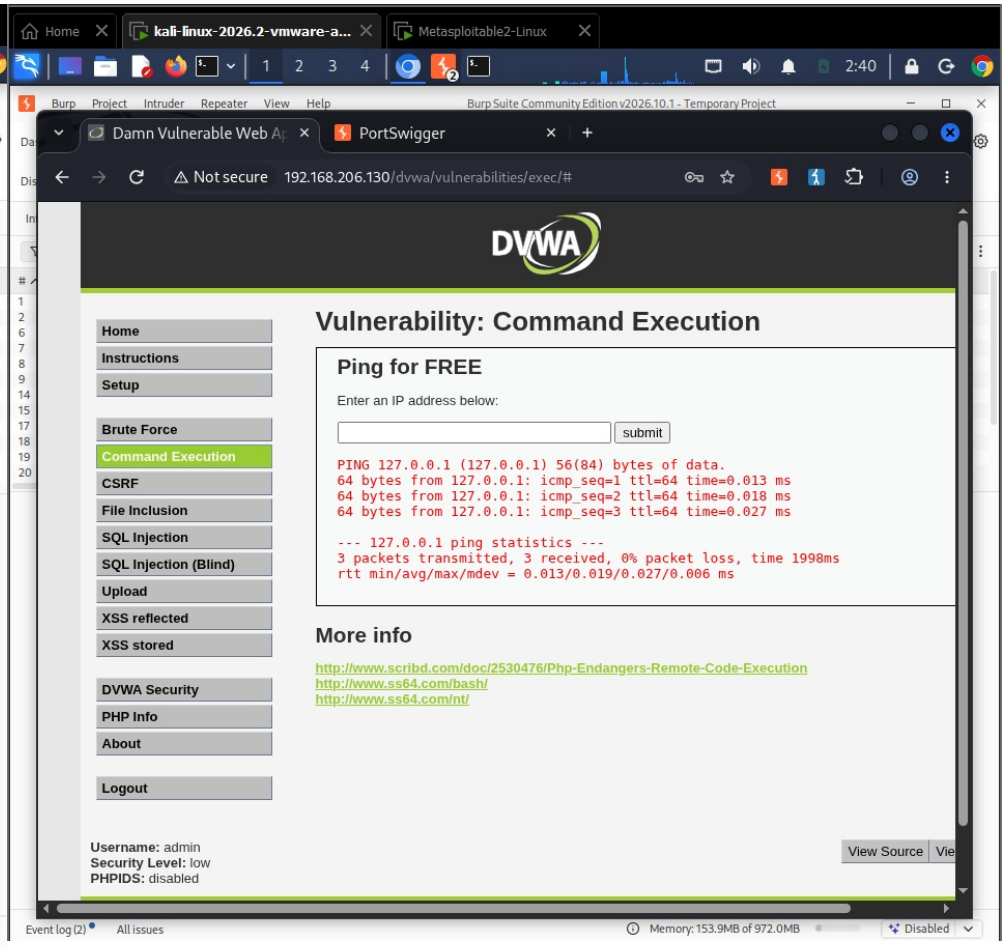
proxy history after manual login to DVWA



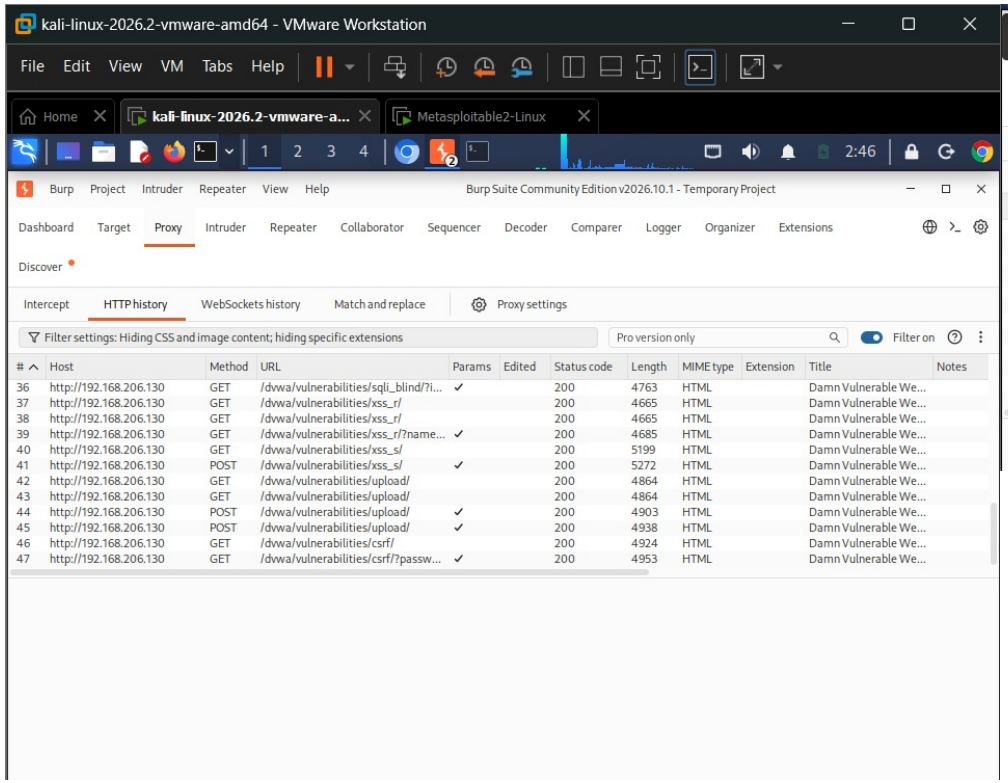
DVWA home page, logged in as admin, through Burp's proxy



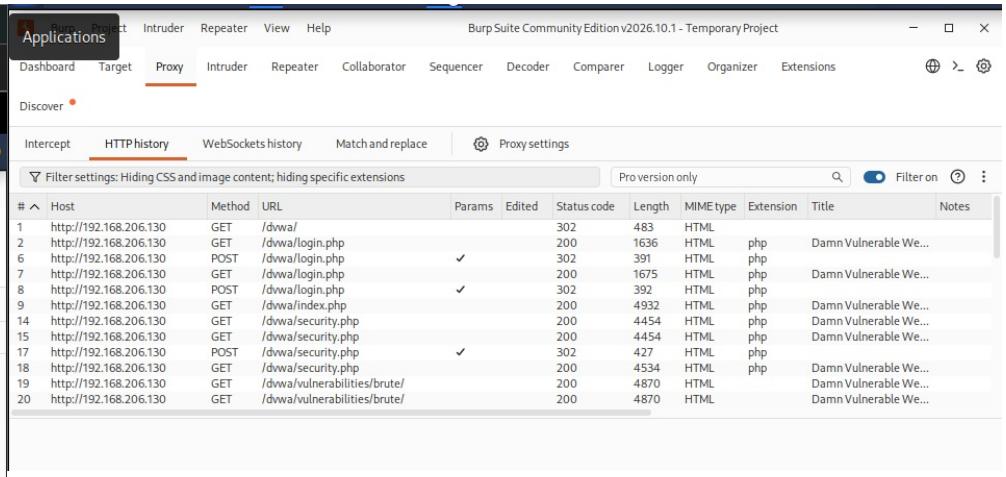
DVWA Security page confirming Script Security: low



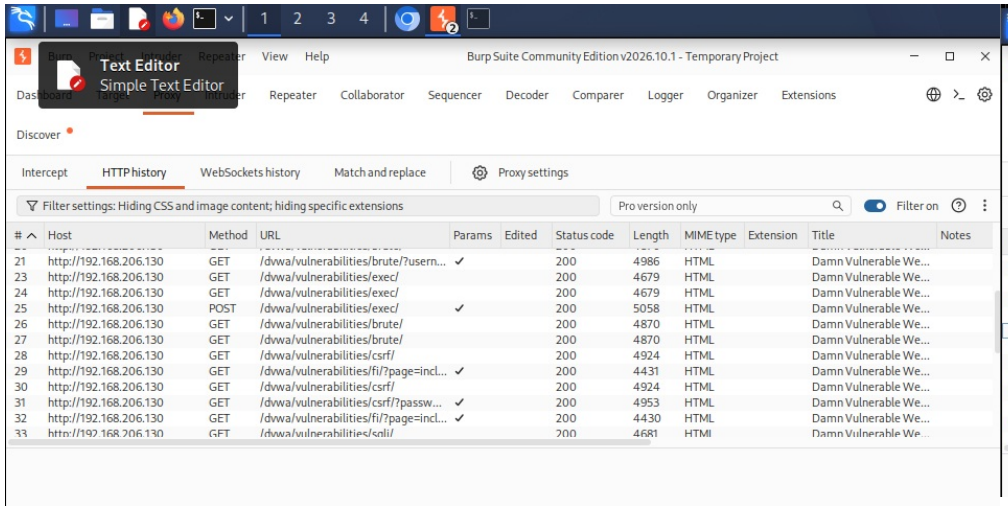
Command Execution page: baseline ping to 127.0.0.1 (no injection yet)



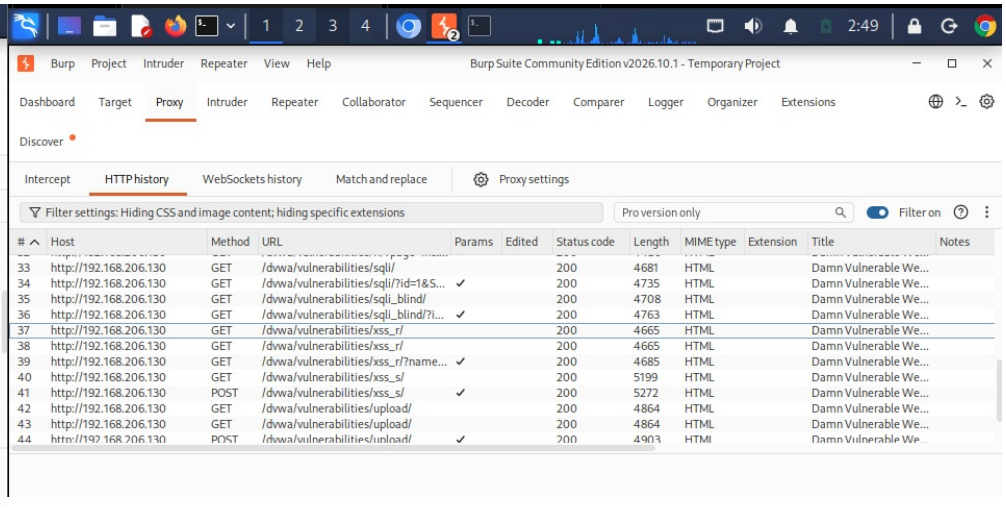
Proxy > HTTP history: requests captured while visiting DVWA Security / Brute Force



Proxy > HTTP history: Command Execution and File Inclusion requests



Proxy > HTTP history: SQL Injection, SQL Injection (Blind), XSS requests

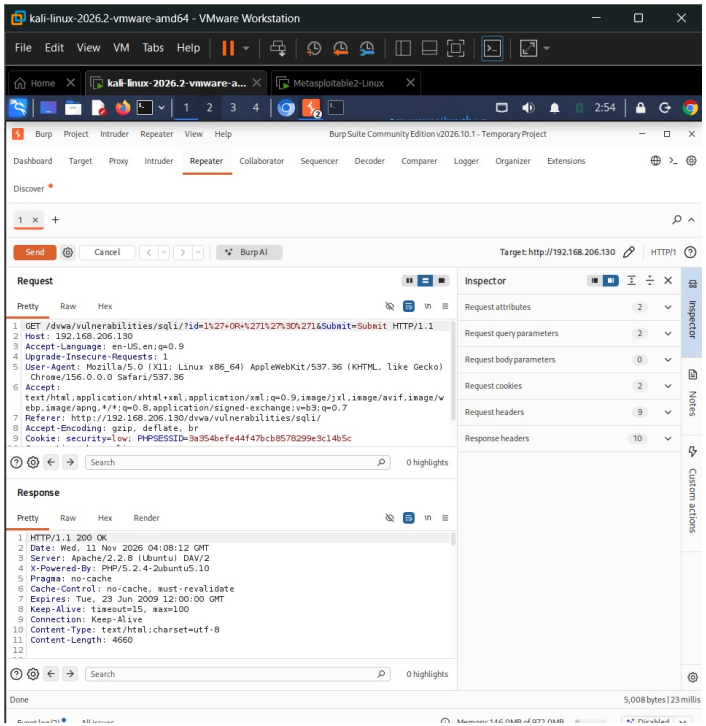


Proxy > HTTP history: Upload and CSRF requests

47 requests were captured in HTTP history after manually visiting and submitting every DVWA vulnerability page (Brute Force, Command Execution, SQL Injection, SQL Injection Blind, XSS reflected, XSS stored, Upload, CSRF, File Inclusion).

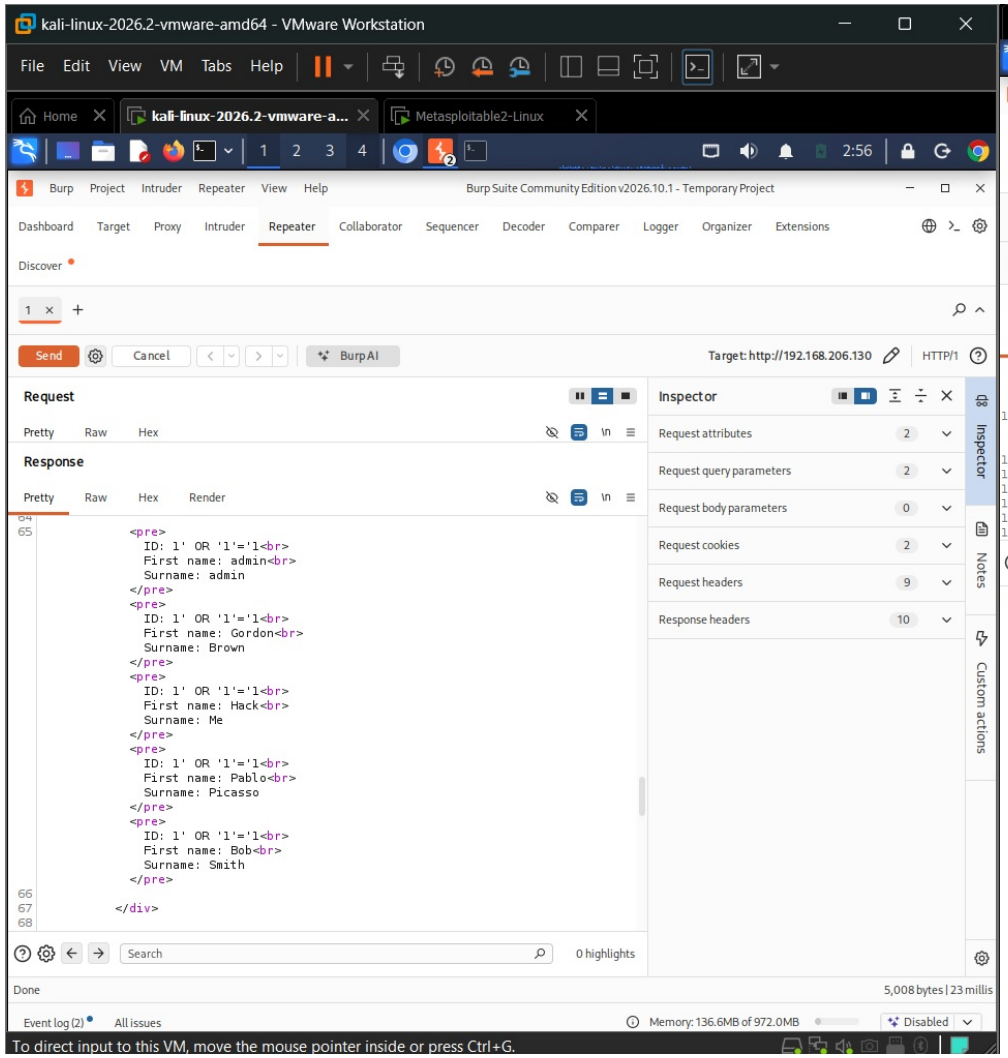
Manual exploitation in Repeater:

SQL Injection – `GET /dvwa/vulnerabilities/sqli/?id=1' OR '1'='1&Submit=Submit` returned all 5 users (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith) instead of the single record for `id=1`, confirming the `id` parameter is concatenated directly into a SQL query with no sanitisation.

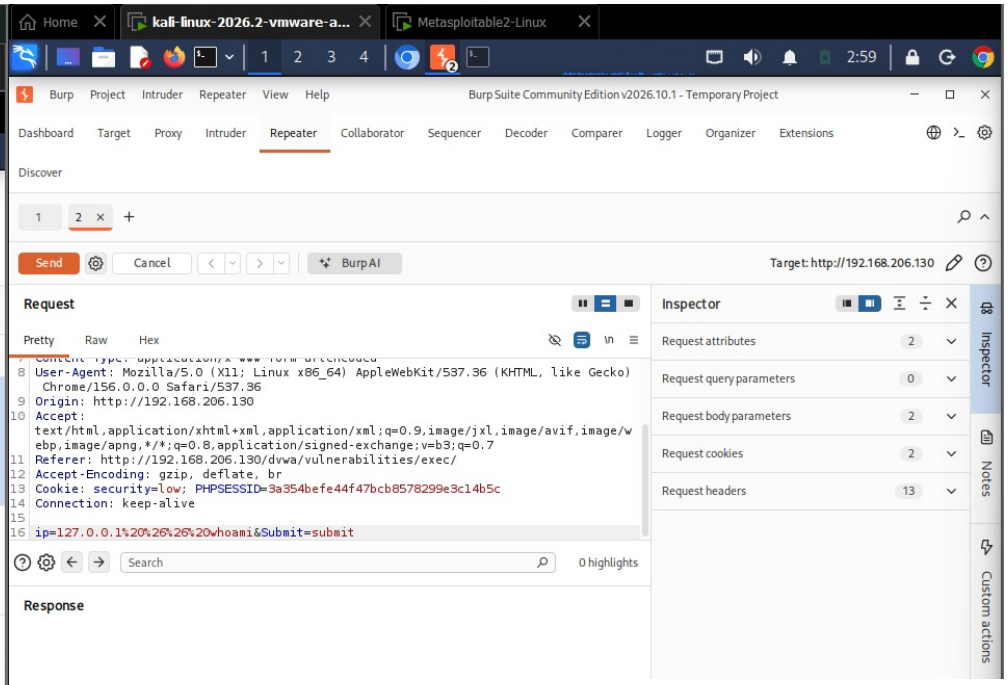


SQL Injection in Repeater: all five DVWA user records returned by the tautology payload

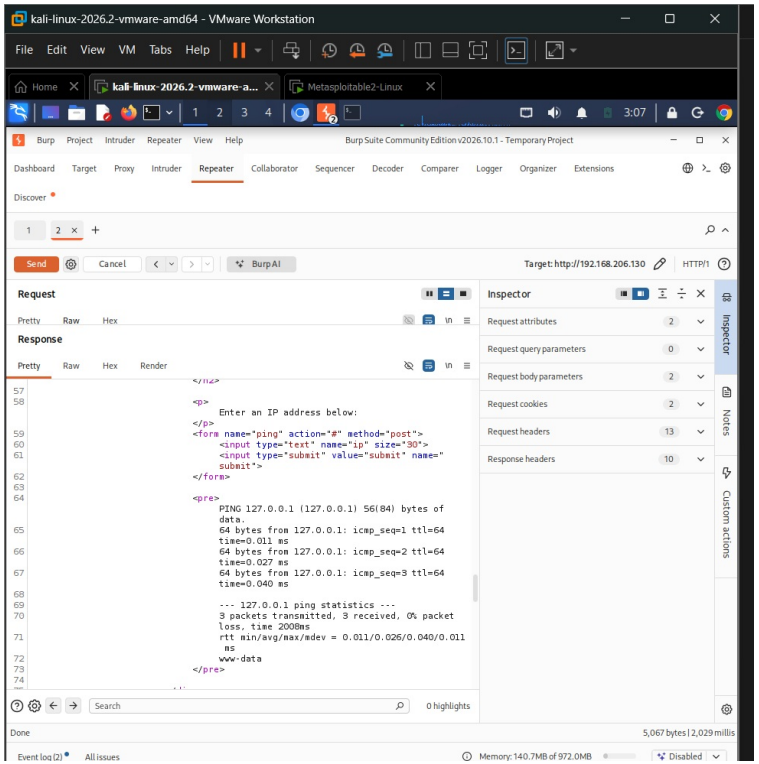
OS Command Injection – `POST /dvwa/vulnerabilities/exec/` with body `ip=127.0.0.1 && whoami` returned the normal ping output followed by an extra line, `www-data` – the result of the injected `whoami` command, confirming the `ip` field is passed straight into a shell command.



first attempt, empty response (debugging)



corrected request (fixed Content-Length mismatch and the parameter name case, submit not Submit)



confirmed: www-data in the response

4.3 – Comparison

	ZAP (Active Scan)	Burp Community (manual)
Found automatically	Path Traversal, RCE (CVE-2012-1823), Source Code Disclosure, CSRF/CSP/clickjacking gaps, cookie flags, header leaks	None – Community's scanner is disabled
Found by manual follow-up	–	SQL Injection, OS Command Injection
Missed	SQL Injection, OS Command Injection (not triggered during automated crawl)	Path Traversal, RCE, header/config issues (no automated scanner to find them)

ZAP's Active Scan, run after a Manual Explore pass, found 3 High-risk issues, 7 Medium, and 5 Low, without manual input beyond the initial browse. It did not report SQL Injection or OS Command Injection – both known to exist in DVWA – most likely because its automated crawl did not submit the vulnerable forms with attacker-controlled input. Burp Community has no automated active scanner at all, so none of ZAP's findings were reproduced automatically; instead, each candidate request was sent to Repeater and tested by hand. Changing the `sqli` page's `id` parameter to `1' OR '1'='1` returned all five user records, confirming SQL injection, and changing the exec page's `ip` parameter to `127.0.0.1 && whoami` caused the server to execute and return an injected command, confirming OS command injection. This shows that automated scanning and manual testing find different, complementary classes of vulnerability: a crawler-based scanner is efficient at path-, file-, and header-level issues it encounters while spidering, while an analyst working by hand through Repeater can confirm injection flaws that depend on submitting specific attacker-chosen values the automated crawl never tried. Building the manual Command Execution request also surfaced a practical lesson in HTTP mechanics: a one-byte Content-Length mismatch after editing the request body silently truncated it, and DVWA's form field is the lowercase `submit` rather than `Submit` – both had to be corrected before the injection would process, illustrating the lower-level precision manual testing in Burp requires compared to a GUI-driven scan.

Repository Contents

All evidence files, screenshots and the raw tool outputs referenced above are published at: https://github.com/the-raven-zn/dfcs-bootcamp/tree/main/s09-web-recon

```
s09-web-recon/
&#9500;&#9472;&#9472; README.md
&#9500;&#9472;&#9472; images/ (53 screenshots, see references above)
&#9500;&#9472;&#9472; subfinder.txt (99 hostnames)
&#9500;&#9472;&#9472; assetfinder.txt (7 hostnames)
&#9500;&#9472;&#9472; amass.txt (7 hostnames)
&#9500;&#9472;&#9472; all_subs.txt (100 deduplicated hostnames)
&#9500;&#9472;&#9472; resolved.csv (DNS resolution results)
&#9500;&#9472;&#9472; live.txt / live_core.txt (httpx-toolkit live-host results)
&#9492;&#9472;&#9472; zap_report.html (ZAP Traditional HTML Report)
```

Note: the .txt / .csv / .html evidence files above were generated on the Kali VM during the lab session and committed to the repository alongside this README.md and the images/ folder.